

Boltzmann Brain: A Versioned, Distributable, and Model-Agnostic Knowledge Architecture

Alex Fiorenza
alexfiorenza2012@gmail.com
Gaussia

 github.com/gaussia-labs/papers

[] gaussia.ai/papers

 huggingface.co/gaussia-labs

Abstract

Knowledge produced with large language models (LLMs) tends to become locked inside a single provider, agent, or vector store, where it is hard to audit, reindex, version, or move. We propose **Boltzmann Brain**, a portable knowledge architecture built on one principle: the brain conserves, validates, and retrieves knowledge, while an external and interchangeable LLM processes, contextualizes, and uses it. The brain embeds no model of its own. Knowledge is stored as typed, content-addressed blocks organized into five memory modules: canonical, episodic, semantic, procedural, and provenance. Each module bundles its blocks, a Merkle DAG that pins the exact composition of a version, and its indices. We separate three levels of hashes, for block identity, module composition, and transportable file, and package each module as a separate OCI Artifact, which yields selective installation and updates at module granularity. Interaction happens through the *Boltzmann Protocol*: the LLM proposes candidate blocks and consumes Evidence Bundles, while the protocol governs what is committed and returns evidence, never prose. Canonical blocks are the root of evidence, recording what was observed, and every derived interpretation cites them through provenance. Wrong knowledge must be removable without punching holes in the current snapshot, so non-episodic modules drop blocks, rebuild their Merkle DAGs, and cascade through provenance, with redaction reserved for destroying retained bytes. This version fixes the interoperability surface, from the block envelope to a conformance kit of golden vectors, and separates integrity from *authenticity*, which hashing alone cannot supply. One detached signature over a snapshot covers every module root and the entire prior history, the authorized keys travel inside the brain, and validity is decided by position in the hash-linked chain, never by a forgeable timestamp. Divergence becomes reconciliation: snapshots name several parents, compositions merge as sets of immutable blocks, and the result is re-validated as an ingestion. We close with the adversarial model this design implies, including what it leaves open.

Keywords: knowledge representation, retrieval-augmented generation, content addressing, Merkle DAG, OCI Artifacts, memory systems, LLM agents

1 Introduction

Large language models have made it cheap to *produce* knowledge (notes, summaries, extracted facts, procedures), but expensive to *own* it. In practice, the knowledge a user accumulates while working with an LLM tends to be captured in a form dictated by whichever provider, agent framework, or vector database happens to be in use: embeddings computed with a specific

model, chunks shaped by a specific pipeline, and a store that is rarely portable, verifiable, or re-interpretable. When the model, the provider, or the indexing strategy changes, that knowledge is often effectively lost, or must be rebuilt from scratch.

This paper argues that knowledge should be a *first-class, portable artifact* that outlives any particular model or vendor. We present **Boltzmann Brain**, an architecture whose sole responsibility is to conserve information, organize it into memories with distinct purposes, maintain verifiable versions, and retrieve relevant evidence. It contains no language model of its own, and the design is captured by a single guiding idea:

The brain conserves, validates, and retrieves knowledge. An external LLM processes, contextualizes, and uses it.

The responsibility boundary. The brain does not write the final answer. It returns knowledge blocks and evidence; an agent, interface, or downstream application decides how to contextualize them (Figure 1). That boundary keeps the architecture independent of any provider: the Boltzmann Protocol governs structure, identity, validation, and distribution, and the semantic work is delegated to an interchangeable external model.

Contributions. This work contributes:

1. Design principles for portable, verifiable, model-agnostic knowledge (Section 2), and an argument for why six requirements that existing work satisfies separately must hold over a single artifact (Section 4).
2. A mapping from biological memory principles and classic systems structures to architectural decisions (Section 3).
3. A five-module memory model, canonical, episodic, semantic, procedural and provenance (Section 5), rooted in the canonical module as observed evidence, and a module anatomy of content-addressed blocks, Merkle DAGs and indices (Section 6).
4. A distribution model over OCI Artifacts giving selective installation and module-granular updates (Section 7), and a model-agnostic protocol with explicit ingestion and query contracts (Sections 9–11).
5. A treatment of how knowledge leaves a brain: drop with Merkle rebuild and provenance cascade, privileged when the excluded evidence is canonical (Section 12).
6. A normative encoding for all of it (Section 6), and a conformance kit of golden vectors (Section 15).
7. A specification of authenticity (Section 8): signatures over snapshots, a trust root that travels inside the brain, signing scopes derived from the protocol’s own operations, quorum-governed rotation of the key list, and revocation by position in the hash-linked chain rather than by a clock.
8. A reconciliation model for diverged brains (Section 13): multi-parent snapshots, a three-way merge over sets of immutable blocks, merge, rebase and squash recast as recording strategies that differ in attribution rather than outcome, and a rule that resolves the remaining conflicts by re-validating the result as an ingestion.
9. A Security Considerations section (Section 14) stating what the protocol guarantees, what it does not, and where the burden sits, including the exposure created by routing preserved evidence into an external model.

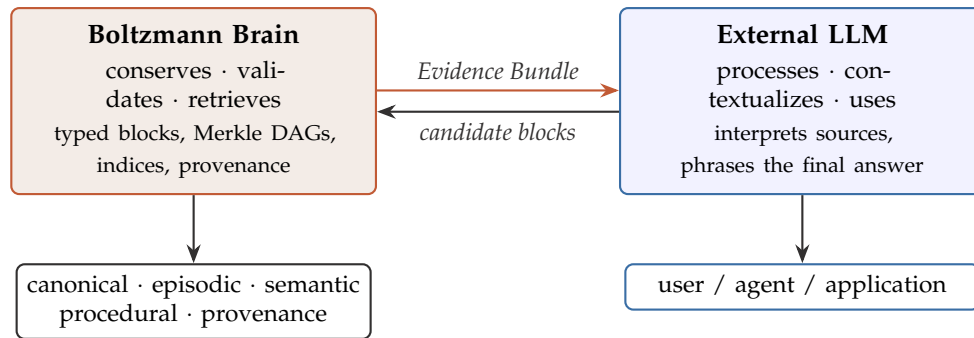


Figure 1: Separation of responsibilities. The brain owns knowledge, identity, and verification; the external LLM owns interpretation and phrasing. The two communicate through a stable contract: the LLM sends candidate blocks during ingestion and receives Evidence Bundles during query.

Scope of this version. This document fixes both an architecture and the interoperability surface that follows from it: everything two independent implementations must agree on to exchange a brain is normative here, and Section 15 states how an implementation demonstrates agreement. What stays deliberately open is what an implementation may choose without breaking exchange: consolidation and contradiction-detection strategies, ranking and fusion, index engines, and pack sizing. The two questions an earlier version left open in the normative surface itself, authenticity of authorship and reconciliation of diverged brains, are specified here (Sections 8 and 13); Section 14 states the exposures that remain, and Section 16 what the protocol still owes, chiefly freshness, which no signature establishes.

Notational conventions. The key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are used as defined in RFC 2119 [5]. A *conforming implementation* is one that satisfies every **MUST** in this document for the roles it claims (Section 15). Documents are shown as JSON for readability; the bytes that are hashed are always the canonical serialization of Section 6.2, never the pretty-printed form.

2 Design Principles and Problems

2.1 Problems this architecture addresses

Boltzmann Brain addresses a cluster of related failures in how LLM-era knowledge is stored and consumed:

- Preventing knowledge from being trapped inside a particular provider, agent, or vector database.
- Maintaining a source of evidence that can be reindexed and reinterpreted as models and methods evolve.
- Distinguishing concrete experiences, general facts, and procedures.
- Updating knowledge without losing history, provenance, or reproducibility.
- Removing obsolete, costly, or harmful knowledge without silently compromising verifiability.
- Distributing a complete brain or only selected modules.
- Letting interchangeable external LLMs query and contribute through a stable contract.

2.2 Guiding principles

Ten principles follow, and they constrain every subsequent design decision:

1. Knowledge lives in blocks; indices are not the source of truth.
2. Original evidence is preserved by default; interpretations may change. Dropping a canonical block is allowed only under explicit policy, and it forfeits re-derivation from that source.
3. Every version must be identifiable, verifiable, and reconstructible.
4. Modules must be distributable and installable selectively.
5. The protocol must not embed an LLM of its own nor depend on a specific provider.
6. Specialized structures compose; no single index answers every query.
7. Queries are declarative and index-agnostic: callers ask for knowledge, and the choice and combination of indices is left to the implementation.
8. Forgetting is explicit and auditable: non-episodic modules support drop with Merkle rebuild and provenance cascade; dropping canonical evidence is privileged and always cascades; content may be redacted under policy, but never silently.
9. Integrity and authenticity are separate claims. Integrity is verifiable by anyone, offline, with no configuration; authorship is a signed assertion evaluated against a trust root that travels inside the brain; and neither claim may ever be reported as the other.
10. Divergence is reconciled, not overwritten. Two brains that advanced from a common ancestor are merged as sets of immutable blocks, and the result is re-validated before it is committed.

These principles are conservative by intent. They privilege durability and auditability over convenience, and treat the LLM as a powerful but replaceable interpreter rather than as the system of record.

3 Inspirations

3.1 Biology of memory

Neuroscience describes several interacting memory systems [16, 24, 27, 37, 48, 49, 51, 61]. The architecture takes *general principles* from that literature, not literal equivalences between brain regions and databases.

Complementary learning systems (CLS) theory distinguishes a fast, hippocampus-associated mechanism from slower, integrative cortical learning [27], so *episodic capture* is separate from *semantic consolidation* here. Hippocampal indexing separates the index that reactivates a memory from its distributed content [16], mirrored in the separation of indices from knowledge blocks. Pattern separation informs deduplication, identity resolution, and contradiction detection; pattern completion, recovering a full memory from a partial query through hybrid search [24, 48, 61]. Reconsolidation makes retrieval an opening for controlled revision [37, 49]. Pruning and active forgetting treat the removal of what is not reinforced as a regulated process rather than a failure, and reduce accessibility without destroying the underlying evidence [11, 45, 51]; Section 12 develops both into concrete retention and erasure mechanisms.

3.2 Systems structures

Alongside biology, the architecture reuses well-understood systems structures: content addressing for block identity, a Merkle DAG so that one root identifies a module’s exact composition, B⁺ trees for exact lookup and ordered range scans, an inverted index for lexical search, a vector index for approximate retrieval by similarity, a graph for explicit relations and provenance, mark-and-sweep collection for reclaiming blocks unreachable from a retained root, OCI Artifacts for packaging and distribution, and a skill, MCP, CLI or API surface as the contract an external LLM speaks.

4 Related Work and the Case for a New Protocol

4.1 What already exists

The industry is already solving pieces of this problem, but no single artifact yet integrates all six of the requirements of Section 4.2. Table 1 positions the most relevant efforts.

Table 1: Related efforts and their distance from Boltzmann Brain.

Project / standard	What it contributes	Distance from Boltzmann Brain
Open Knowledge Format	Markdown + YAML as a portable, human-readable format for agents.	Formalizes canonical exchange, but defines no access protocol, biological memory model, indices, or compiled distribution.
GBrain	Git repository as a durable registry; hybrid search, graph, and agent access.	Reinforces the independence of knowledge base from agent; its primary distribution follows the Git/repository model.
Microsoft GraphRAG	Extracts entities, relations, communities, and summaries; combines graph and text.	An indexing and retrieval pipeline, not a distributable format for a complete brain.
Cognee	Turns data into chunks, embeddings, nodes, and relations; combines vector and graph.	Provides memory operations and enrichment, but does not define the proposed OCI/Merkle bundle.

The Open Knowledge Format (OKF) formalizes the knowledge-wiki pattern through Markdown and YAML, emphasizing independence between producers and consumers [17]. GBrain separates durable knowledge from agent behavior and offers interchangeable engines and agent integration [55–57]. GraphRAG and Cognee illustrate the convergence toward hybrid representations that mix text, vectors, and graphs [9, 10, 31, 32].

The older families this design descends from. The closest prior art predates every row above. *Nanopublications* [19] package one assertion with structurally mandatory provenance as a minimal citable unit, and their *trusty URIs* [25] are content hashes over a normalized serialization, tagged with a versioned algorithm identifier, the same move the envelope makes with serialization; the identifier’s range of verifiability is explicitly recursive, so a tree of *trusty URIs* carries its history the way hash-linked snapshots do, and nanopublication *indexes*

name sets of units by reference rather than containment, which is the composition document's shape [26]. Where the family stops is where this design begins: normalization is defined at the RDF-graph level rather than over bytes, there is no Merkle root and no $O(\log n)$ membership proof, an index appends to at most one predecessor where parents is a list, and the trusty-URI specification defines no signature mechanism (later nanopublication practice attaches one), so scoped keys, quorum rotation and positional revocation have no analogue. On provenance vocabulary, W3C PROV [30] is the standard; the six closed record types of Section 5 trade its generality for records a verifier can check mechanically, and a deployment that needs interoperable provenance can express them in PROV terms without loss.

Memory typing is a convention, not a contribution. On the typing axis this paper claims no novelty. The episodic/semantic distinction is Tulving's [60], the cognitive architectures added procedural memory and store all three as separate long-term memories [28], and CoALA imports exactly that typing as a design framework for LLM agents, surveying 2022–2023 systems that already implemented typed writable memory [54]. What this design adds on the axis is the two modules the convention does not carry, canonical evidence and closed-form provenance, and the verifiable representation all five share.

Concurrent work. Two 2026 preprints occupy the nearest ground and predate this paper by weeks. Portable Agent Memory [47] defines a five-component typed memory whose entries are content-addressed (BLAKE3 over a canonical JSON the format itself defines), linked into a Merkle-DAG provenance graph, and covered by a single Ed25519 operator signature, with capability-scoped disclosure and an injection-aware rehydration procedure. It is the closest published design, and the distance is the verification and distribution surface: it specifies no standardized tree construction (no RFC 9162 Merkle Tree Hash, per-module roots, or inclusion proofs), no standardized canonicalization, no distribution encoding (no OCI artifact, selective installation, or history transport), and no key governance (no scoped trust root, quorum rotation, positional revocation, or reconciliation semantics), and its retention model does not reach drop cascades or redaction. memorywire [36] is a vendor-neutral wire format for five memory operations over four memory types, motivated by the same absence of a shared format; its version 0 defines no content addressing, hashing, or signing, and lists signed envelopes and audit chains as roadmap items. Both corroborate the diagnosis of Section 2 rather than deliver the artifact this paper specifies: the conjunction of canonical evidence, a normative Merkle construction with a named layout, scoped and quorum-governed authenticity, and module-granular OCI distribution remains unclaimed by either.

Substrate rather than alternatives. Two standards that recur throughout this paper are deliberately absent from the comparison. MCP standardizes how applications connect to external resources and tools [2], and the OCI ecosystem provides content-addressed manifests, descriptors, and blobs whose structure is itself an external Merkle DAG [39, 40, 44]. Neither is an alternative to a knowledge architecture, because one standardizes a call and the other a transport, and this design adopts both instead of competing with them (Sections 7 and 9).

4.2 Six requirements that must hold simultaneously

Proposing a new protocol requires more than observing that existing tools are imperfect. The argument of this paper is that the problems of Section 2 translate into six requirements which, to be useful, must hold *at the same time and over the same artifact*:

R1. Canonical evidence. The original source is preserved and addressable, so that any interpretation can be traced to the bytes it cites and re-derived when methods improve. Without this, knowledge cannot be re-interpreted, only trusted.

- R2. Typed memories.** Concrete experiences, consolidated facts, and procedures are distinguished rather than collapsed into undifferentiated chunks. Without this, “what happened in the class of May 14” and “the definition of a Fourier series” compete in a single similarity ranking.
- R3. Verifiable versioning at knowledge granularity.** Versions are identifiable, verifiable, and reconstructible at the granularity of a unit of knowledge rather than a file. Without this, a corrected fact cannot be distinguished from an edited document.
- R4. Derived, replaceable indices.** Indices are views that can be rebuilt or replaced when an engine or embedding model changes. Without this, knowledge dies with the model that indexed it.
- R5. Selective, module-granular distribution.** A consumer can install one module, and an update re-transfers only the modules that changed. Without this, sharing knowledge means shipping everything, every time.
- R6. Model-agnostic access contract.** A stable, declarative contract lets interchangeable models query and contribute. Without this, the store is reachable only through whoever wrote the integration.

Table 2 scores the efforts of Table 1 against these six axes. The pattern is not that existing work is weak; it is that each effort is strong on a different axis. Portable formats concentrate on R1 and leave retrieval undefined, while retrieval pipelines concentrate on R4 and treat versioning and distribution as someone else’s problem. The closest neighbor, GBrain, is partial on almost every axis precisely because it inherits its versioning and distribution from Git rather than defining them at the knowledge layer. What no row does is combine more than a fraction of the six, which is the gap this paper addresses.

Table 2: Design coverage of the six requirements of Section 4.2: ● full, ◐ partial, ○ absent. Marks in the last row are grounded in the golden vectors of Section 15 and the reference implementation rather than in design intent, and R1 is partial by the protocol’s own admission: cited evidence is verified to exist and be addressable, while whether a claim holds over it is not checked (Section 11.3).

Project / standard	R1	R2	R3	R4	R5	R6
Open Knowledge Format	●	◐	◐	○	○	○
GBrain	●	◐	◐	◐	◐	◐
Microsoft GraphRAG	◐	◐	○	◐	○	○
Cognee	◐	◐	○	◐	○	◐
Vector store + RAG pipeline	○	○	○	◐	○	◐
Boltzmann Brain (this work)	◐	●	●	●	●	●

4.3 Why composition is not enough

Every one of the six requirements is met somewhere in the wider ecosystem, once the standards of the previous subsection are counted, so the obvious objection is that the answer is integration rather than a new protocol: put the sources in Git, run GraphRAG over them, expose the result through MCP, and ship it with OCI. That stack is worth building, and much of it is what a conforming implementation would look like. What it does not produce is a portable artifact, for four reasons.

Identity does not compose. Each layer carries its own notion of what a thing is: a commit, a pipeline output, a registry digest. None of them binds a retrieved item to the source it came from and to the snapshot it belongs to. That binding is what an Evidence Bundle asserts when it reports verified, and why this architecture separates three levels of hashes (Section 6) instead of borrowing one. Verifiability is a property of the whole path from source to answer, so it cannot be added by a layer that only sees its own inputs.

Granularity mismatch. Version control versions files; knowledge changes at the granularity of a claim. A corrected definition, its evidence, and the procedures that depended on it are not a diff over lines, and recovering that structure from a file history means reconstructing what the ingestion step already knew. R3 asks for versioning where the knowledge is, which is the block.

The layering runs the wrong way. In a retrieval pipeline the index is the product: documents are consumed to build embeddings and graphs, and those derived structures become the thing that is queried, stored, and eventually migrated. R4 requires the opposite ordering, with blocks as the durable artifact and indices as disposable views over them. Inverting an existing pipeline is not a matter of configuration, because a pipeline whose output is its state has no representation of the knowledge that survives it.

Access and transport standardize the edges, not the middle. MCP standardizes the call and OCI standardizes the bytes. Neither says what is on the other side of the call or inside the blob. Two MCP servers over the same corpus share no verifiable state, and two registries holding the same brain agree on digests while knowing nothing about modules, memory types, or snapshots. The missing piece sits between them: a format for typed, content-addressed blocks plus a contract for reading and extending them.

Why now. The same argument would have been premature a few years ago, when the model, the pipeline, and the application were chosen once. Today embedding models are deprecated on a timescale of months and agent frameworks turn over faster still, so accumulated knowledge routinely outlives the tools that produced it. The container ecosystem went through this transition: engines and registries became interchangeable only once image identity and packaging were specified independently of any implementation. LLM knowledge is at the earlier stage, with capable engines and no shared artifact, and this paper argues for specifying the artifact.

Complementarity. Boltzmann Brain is therefore complementary rather than competing: it can consume OKF-style canonical sources, borrow hybrid retrieval ideas from GraphRAG and Cognee, expose an MCP interface, and ride on OCI for distribution, while contributing the missing piece, a versioned, typed, model-agnostic *bundle* whose modules install and update selectively and whose contents stay verifiable no matter which model reads them.

5 Knowledge Modules

The brain's OCI Artifact can contain five logical modules. Each module is installable as an independent physical unit and contains blocks, a Merkle DAG, and the indices needed for query. Table 3 names each module by the question it answers.

Table 3: The five memory modules, illustrated with a university brain.

Module	Question it answers	Examples
Canonical	What material was observed?	Notes, books, PDFs, lectures, code, datasets.
Episodic	What happened in a concrete context?	Class of May 14, an error in an exercise, a professor’s correction.
Semantic	What general knowledge was consolidated?	Definitions, formulas, facts, relations.
Procedural	How is a task performed?	Solving an integral, computing Fourier coefficients.
Provenance	Where did it come from and how was it transformed?	Page, source, pipeline, model, validator, date.

Canonical module. Preserves observed evidence, not truth claims. A canonical block asserts that a source was incorporated and that it *contains* certain bytes; it does not declare those bytes correct. Because every semantic and procedural interpretation cites canonical evidence through provenance, this module is the root of re-derivation: without the source, derived knowledge cannot be rebuilt cleanly from below.

The module holds two content-addressed layers. The *original* is the blob as observed (PDF, repository archive, conversation export, image, audio, and so on). An optional *normalized view* is a deterministic transform of that blob (plain text, Markdown, or a structured extract) produced by a named pipeline whose identity is recorded in provenance. Normalized views live in canonical rather than semantic memory because they are still evidence for ingestion, not consolidated knowledge. Both layers are addressed by hash.

A canonical block therefore carries only what its bytes determine: its `block_id`, media type, size, and optionally the digest of its normalized view. It carries nothing else, so that re-registering an identical original is a no-op: if the registering actor or the moment of registration entered the hashed content, two people incorporating the same source would obtain different blocks and deduplication would never trigger. Everything historical or actor-dependent is recorded in the provenance module instead: the license or retention policy under which the source is held, the actor who incorporated it, the timestamp, and the supersession edge toward an earlier edition when a new one replaces it (Sections 10 and 12).

Episodic module. Records events and experiences with time, context, participants, outcome, and evidence. It is the only module that stays append-only: corrections generate new episodes or supersession relations rather than drops that rewrite the past (Section 12).

Semantic module. Holds consolidated general knowledge: concepts, formulas, facts, relations, and constraints. A Fourier-series formula belongs primarily here, linked to its canonical sources.

Procedural module. Represents action sequences, conditions, decisions, alternative paths, and success criteria, for example how to obtain the Fourier coefficients of a given function.

Provenance module. Connects derived blocks to evidence, transformations, and tool versions. It answers what must be recomputed when a source changes, and it is the ledger of removals: every drop, cascade, and redaction is recorded there so that exclusion stays auditable (Section 12). The actor named in a provenance record is a *declared* identifier; the signature over the snapshot that committed the record is what turns it into an authenticated identity (Section 8).

Provenance is not free-form annotation. It carries seven kinds of record (Table 4), and the set is closed because every other part of the protocol depends on being able to ask a precise question of it: which blocks cite this evidence, what produced this block, what verdict admitted it, and what was removed by whom.

Table 4: The provenance records, and what each one makes answerable.

Record	What it states
Registration	A source was incorporated: by which actor, when, from where, and under what license or retention policy.
Normalization	A normalized view was produced from an original by a named deterministic pipeline.
Derivation	A block was derived from cited evidence by a named producer, within a named task.
Supersession	A block takes precedence over an earlier one.
Demotion	A block's retrieval priority was reduced, without removing it.
Validation	A committed block received its verdict: which checks ran and within which task.
Removal	A block was excluded, pruned, or redacted: by which mechanism, actor, and reason.

Who a record names. An *actor* is who performed an operation. Every record names exactly one, and a record MAY additionally name the parties that *assisted*: the people and the agents that took part without being the one who performed it. Most brains are hydrated through an agent, so the assisting parties are usually where the work actually happened, and a ledger that could not name them would record the operator and lose the author.

One question this protocol does not ask

Nothing here assigns responsibility. There is an actor, there is whoever assisted, and the protocol takes no position on which of them answers for the knowledge. Responsibility is a matter for the deployment, the jurisdiction, and the people involved; a field claiming to settle it would be a field every implementation would have to interpret, and none of them would agree.

Actor identifiers. An actor identifier MUST take one of exactly two forms: an *addr-spec* as defined by RFC 5322, or a *namespaced name* namespace/name. It MUST be lowercase, and an implementation MUST reject a non-conforming identifier rather than normalize it, for the reason canonical serialization exists at all: an identifier that two parties spell differently is two identifiers, and because provenance records are blocks, two spellings of one person are two *block_ids* for one fact (Section 6.1).

No scheme prefix is stored. A `mailto:` or `https:` wrapper is ceremony around a value people already write correctly, and the normalization questions it invites — a trailing slash, a default port, a percent-encoded octet — are exactly the questions a canonical form must not leave open. An implementation exporting to a provenance format that requires URIs derives one at that boundary.

The kind of actor is recorded alongside the identifier and does not constrain its form. A contributor known only by a handle is precisely the party Section 13.6 is written for, and requiring an address would close a door the protocol deliberately leaves open.

Table 5: The two identifier forms, and what each is for. The namespace carries the party that created or vouches for the name, so no separate field repeats it.

Form	Names
alex@example.org	A person, by an address they already hold.
example.com/alex	A person with only a handle, or an organization.
anthropic/claude-code	A runtime: the harness an agent runs inside.
anthropic/fable-5	A model.

Assisting parties, and what a model ran. An assisting party is named the same way, and MAY additionally name the *model* it ran. The pair matters: the same model under a different harness is a different collaborator, because the harness decides what the model sees, how many turns it takes, and which tools it can reach. Naming the two together keeps the pairing unambiguous when several agents write into one snapshot. A party of human kind MUST NOT name a model.

No version string appears in any of this. A version is the field most likely to be invented by whoever fills the record in, it ages faster than everything around it, and it buys less than the identity it sits beside.

Producers, and what became of them. A *producer* is what generated a derived block, recorded at the granularity a batch invalidation needs: a model, a pipeline, an ingestion batch, or an actor. It is the older spelling of the same question, and derivation records at schema version 1 carry it. From schema version 2 it is replaced by the assisting parties, which answer it more completely and for every record kind rather than for derivation alone. The batch invalidation of Section 12.3 is a query over whichever of the two a record carries.

6 Anatomy of a Module

A module carries three responsibilities in the same physical unit: *blocks* contain knowledge; the *Merkle DAG* defines which blocks form a version; *indices* make it possible to find blocks without scanning all of them.

6.1 Knowledge blocks

A block is a logical unit of knowledge with a deterministic serialization. It may be an episode, a concept, a fact, a relation, a procedure, or a canonical fragment. Its identifier is computed from its content:

$$\text{block_id} = \text{SHA-256}(\text{canonical_serialization}(\text{envelope})). \quad (1)$$

If the content changes, the result is a new block; the previous one may be kept for history, audit, or rollback.

The envelope. What is hashed is not the payload alone but an *envelope* that carries exactly five keys:

```
{
  "boltzmann":      1,
  "memory_type":    "semantic",
  "payload":        { ... },
  "schema_version": 1,
  "serialization":  "jcs/1"
```

}

A conforming implementation **MUST** reject an envelope carrying any other key, or missing any of these. Binding the memory type and the schema version into the hash makes identity self-describing: the same payload read as an episode and as a semantic fact is two different blocks, and stored bytes can be decoded without consulting anything outside them.

Everything derived, physical, or actor-dependent stays outside the envelope: the storage location, the index state, and the moment of registration are not part of what a block *is*. This is the same argument that keeps canonical blocks free of registration metadata (Section 5), applied uniformly.

Two conventions that keep identity unambiguous. An absent optional field **MUST** be omitted from the payload rather than serialized as `null`, so that `{"a":1}` and `{"a":1,"b":null}` are the same block rather than two blocks with the same meaning. Blocks **MUST** be treated as frozen: correcting a block means creating a new one.

Content a block names but does not carry. A payload is JSON, so a block whose datum is large or binary (a PDF, an audio file, a repository archive) names it by digest and leaves the bytes in the store. A block therefore has a set of *content digests*, possibly empty. Every operation that must account for those bytes, namely packing a layer, marking reachability before a prune, and destroying bytes on redaction, **MUST** consult that set rather than inspect the block's type, so that a schema which starts naming content is handled correctly by all of them at once.

Decoding. A conforming implementation **MUST** verify, when decoding stored bytes, that re-serializing the decoded block reproduces those bytes exactly. Bytes that are not already in canonical form would hash to a different `block_id` than the one under which they are stored, so they **MUST** be rejected rather than silently normalized.

6.2 Canonical serialization

Two clients that disagree on serialization compute different identifiers for the same knowledge, and then deduplication never triggers and two parties holding identical knowledge cannot tell that they do. The serialization is therefore normative, and it is named in every envelope so that a future one can coexist with it rather than replace it.

This version uses **JCS**, the JSON Canonicalization Scheme of RFC 8785 [50]: object keys sorted by UTF-16 code unit, no insignificant whitespace, and a fixed number format. Its identifier is `jcs/1`. JCS was chosen over a binary encoding because a block is a small structured record and the protocol is meant to be implemented in several languages: a canonical form that a human can read and `grep` is worth more here than compactness, and RFC 8785 has independent implementations across the language ecosystems a client is likely to be written in.

Identity and signature verification are different dependents. Canonical bytes do two jobs here, and adjacent standards object to one of them. For *identity* there is no alternative: `block_id` exists so that two parties who registered the same knowledge compute the same name, and that is what a canonical form is for. For *signature verification*, DSSE and the in-toto attestation framework recommend against it: an envelope should avoid depending on canonicalization for security and should not require the verifier to parse the payload before verifying, because a parser disagreement between two payloads sharing one canonical encoding becomes a signature bypass [20, 53]. C2PA answers the same objection by constraining the producer instead: claims are serialized under RFC 8949's deterministic-encoding requirements and signed as stored, with no verifier-side re-encoding [8]. This design lands on that side of the line. Canonical form is

an encoder obligation; stored and transported bytes are the canonical bytes (Section 6.1, and Section 7 for the config blob); and a verifier checks canonicity and then hashes and verifies the bytes as stored, never trusting a parse-and-re-serialize round trip for the answer.

Two value domains are excluded from a payload

JCS defines float serialization through ECMAScript rules that are subtly hard to reproduce identically across languages, and integers outside the IEEE-754 safe range ($\pm(2^{53} - 1)$) lose precision in any JSON parser backed by doubles. Either divergence would mean two conforming clients computing different `block_id` values for the same knowledge. Both MUST therefore be rejected when a block is constructed, rather than at hashing time. Represent a decimal as a string, or as an integer scaled by a documented factor.

Three precisions RFC 8785 leaves to its caller. Key order is by UTF-16 *code unit*, not code point: outside the Basic Multilingual Plane the two orders differ, because surrogates sort below the code points they encode, and an implementation sorting by code point silently computes different identifiers for the same knowledge. A JSON object carrying the same key twice MUST be rejected when a block is constructed: RFC 8785 presupposes I-JSON input, parsers disagree on which duplicate survives, and either choice silently changes the bytes. Input that is not valid UTF-8, or that contains lone surrogates, MUST be rejected for the same reason. And JCS does not normalize Unicode: a payload in NFC and the same text in NFD are different bytes, and therefore two blocks with one meaning. This version keeps it that way, because normalizing inside the identity computation would mean the stored bytes are not the bytes that were registered; a producer SHOULD normalize text to NFC before constructing a block, and the burden sits with the producer.

6.3 Merkle DAG

On their own, blocks are a bag of content-addressed units: each is addressable by its hash, but nothing names, verifies, or diffs a whole *version* of the module. The Merkle DAG stores no meaning by itself; it maintains references by hash and produces a single root that commits to the exact set and structure of blocks in a version. That root *is* the identity of the version, and it makes Principle 3 concrete.

How it works. The leaves of the DAG are `block_ids` (content hashes); each internal node hashes the hashes of its children; the root hashes down to everything below it. Changing any block changes its hash, which changes every ancestor up to the root. Three operations follow. *Integrity* is checked by recomputing hashes from the leaves up and comparing the root. *Membership* of a single block is proven with the sibling hashes along its path to the root (a Merkle proof of size $O(\log n)$), without downloading the rest of the module. A proof MUST carry the block's leaf index and the tree size, because the two are verifier inputs rather than metadata: the tree's shape is a function of its size, so which side each sibling combines on, and where the path terminates, are decided by them, and a bare sibling list is ambiguous between roots. RFC 9162's verification algorithm takes both [29]. *Differencing* two versions is a set operation over their sorted leaf lists, linear in the size of the module and cheap at that size, since each leaf is a single digest. The tree is needed for the first two operations only, which is why nothing beyond the leaves and the root is persisted.

The construction is normative. Roots are only comparable between clients that compute them the same way, so the construction is fixed rather than left to an implementation. A conforming implementation MUST use the Merkle Tree Hash of RFC 9162, Section 2.1.1 [29], over the

module's block identifiers, with two properties that matter for security and for the guarantees claimed above:

- **Domain separation.** Leaves are hashed with a `0x00` prefix and internal nodes with `0x01`, so a leaf hash can never be mistaken for a node hash; RFC 9162 attributes second-preimage resistance to this separation.
- **Unambiguous splitting.** The tree splits at the largest power of two below n . The naive alternative, duplicating the last node on an odd level, is the defect behind CVE-2012-2459, in which two distinct leaf lists produce the same root [34], defeating the whole point of identifying a version by its root.

Leaves are sorted. Leaves MUST be deduplicated and ordered by their digest before the tree is built. Sorting makes the root a function of the *set* of blocks, which is the claim that two parties who assembled the same blocks obtain the same root. A layout that preserved insertion order would break that claim while looking correct in every single-writer test. Both are producer obligations, and a verifier MUST enforce them: a composition document whose `block_ids` are not in strictly ascending digest order MUST be rejected, exactly as an unknown layout is (below). Without that check one leaf set admits many roots, since every permutation yields a different, internally consistent root; that is equivocation, and it defeats the claim the sorting exists to make. Strictly ascending order enforces deduplication in the same comparison, and it fixes what the root commits to: a set rather than a multiset, with multiplicity deliberately not representable.

The layout identifier. The construction is named `rfc6962-sorted/1`, and that identifier travels with every composition and module reference. A client that meets a composition built under a layout it does not implement MUST refuse it rather than recompute a root that would not be comparable. The name keeps its historical form: RFC 9162 obsoletes RFC 6962 but defines the same tree, with the same empty hash, the same prefixes, and the same split, so a root computed under either reading is byte-identical. The suffix moves if the construction ever does. The sorted, deduplicated leaf list is this specification's own modification of the RFC's construction, with no published analysis behind the reordering to lean on, which is why the ordering check above is a verifier obligation rather than a convention.

What is persisted. Only the sorted leaf list and the root are stored. Internal nodes are scaffolding derived on demand, so there is nothing to keep in sync, and differencing two versions is a set operation over leaf lists rather than a descent through stored nodes.

Why a DAG rather than a tree. Because blocks are shared across versions. When a new version reuses unchanged blocks, those blocks are shared by hash instead of duplicated, so a node may be referenced by more than one root. The structure is therefore a directed acyclic graph, as in Git's object store. This structural sharing makes new versions cheap to store: a version adds only the changed blocks and a new root, and reuses everything else by reference. Sharing guarantees the reuse of blocks in local storage; what an update re-transfers is decided by the distribution encoding, whose granularity is the module (Section 7). How much of the structure above the leaves is reused depends on the layout an implementation chooses, and internal nodes carry no content of their own.

Two Merkle DAGs. It is worth separating the *internal* Merkle DAG described here, which versions the blocks inside a module, from the *external* Merkle DAG that OCI itself forms over manifests, descriptors, and blobs to version whole modules (Section 7, Figure 4). The same idea is applied at two levels: blocks within a module, and modules within a brain.

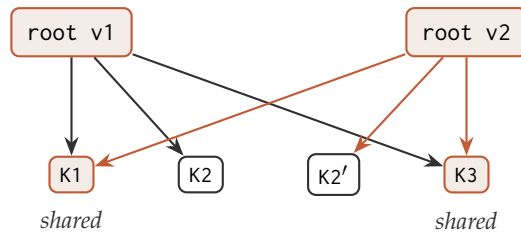


Figure 2: Structural sharing in the internal Merkle DAG. Correcting K2 into K2' yields a new root v2 that reuses K1 and K3 by hash; only the changed block and the new root are added. Solid dark edges belong to v1, accent edges to v2.

Storage-independent identity. Because the root depends only on the blocks and their structure, it is a canonical identity of a knowledge state, independent of how that state is stored or transported: two parties that assembled the same blocks obtain the same root. The OCI digest, by contrast, identifies a particular physical file or manifest.

6.4 The composition document

A Merkle root commits to a set of blocks but *cannot be inverted back into it*, so a version identified by its root alone can be verified and not reopened. Given a root, a client can confirm that a leaf list it already holds is the right one, but it cannot recover that list from the root. Something has to store the leaves.

That something is the *composition document*: the persisted set of block identities that a root commits to. It carries the protocol version, the memory type it belongs to, the Merkle layout that produced the root, and the leaf list in canonical order:

```
{
  "boltzmann": 1,
  "memory_type": "semantic",
  "layout": "rfc6962-sorted/1",
  "block_ids": ["sha256:K1", "sha256:K2", "sha256:K3"]
}
```

Storing it is not redundant with the root: the root is the *claim* about a version, and the document is what makes the claim resolvable. This document is what a module layer carries when a brain is published (Section 7), and it is the object every removal mechanism actually operates on. A drop does not mutate a block; it produces a new composition, and therefore a new root (Section 12.3).

A composition **MUST** be immutable: every operation over it yields a new composition rather than modifying one in place, so that a root already handed to a consumer can never change underneath it. A conforming implementation **MUST** refuse a composition document whose declared layout it does not implement, and **MUST** refuse one whose declared protocol version it does not implement.

6.5 The snapshot

A module root identifies one module. A brain is several modules at once, and nothing said so far names the combination. The *snapshot* is that document: it states, for each installed module, which version of it this brain currently is.

```
{
  "boltzmann": 1,
```

```

"modules": {
  "canonical": { "root":          "sha256:AAA",
                 "composition":   "sha256:AAAdoc",
                 "block_count":   412,
                 "layout":        "rfc6962-sorted/1" },
  "semantic": { "root":          "sha256:CCC2",
                 "composition":   "sha256:CCC2doc",
                 "block_count":   1907,
                 "layout":        "rfc6962-sorted/1",
                 "embedding_model": "bge-m3@1.5",
                 "index_digest":  "sha256:III2" },
},
"created_at": "2026-07-24T10:15:00Z",
"parents":    ["sha256:PREVIOUS_SNAPSHOT"],
"trust_root": { ... authorized keys and their scopes ... },
"labels":     { "release": "v8" }
}

```

Each module reference carries the root (its logical identity), the digest of the composition document (so the version can be reopened, not merely verified), the block count, the layout, and, when a vector index travels with the module, the embedding model and version that produced it together with the digest of the index payload, which is what places a travelling index inside the signed bytes (Section 6.6). A module whose composition still names destroyed bytes additionally lists those identities in a tombstones member, which is what lets a verifier see a redaction in the difference between snapshots (Section 12).

Five properties do most of the work in the rest of the paper.

It is the unit of atomic change. Adding one semantic block also advances provenance, and a canonical drop can advance three modules at once (Section 12.3). Those are still *one* version of the brain. A conforming implementation **MUST** advance every affected module in a single successor snapshot rather than one at a time: minting intermediate snapshots that were never published would leave the parent chain pointing at documents no consumer can resolve.

It is a chain, and occasionally a graph. The parents field names the snapshots this one succeeds, so the history of a brain is an auditable hash-linked structure rather than a sequence of unrelated roots. This is what a consumer walks to tell whether a remote brain is ahead of, behind, or diverged from a local one (Section 7.4). A linear history carries one entry, which is the ordinary case; a first snapshot carries none; a reconciliation of two histories carries two or more (Section 13). The list is ordered, and the *first* parent is the one every rule elsewhere in this document means by “the parent” (Section 13.1).

It is a subset, not necessarily the whole. A brain may hold only some modules, because selective installation is the point of packaging each module separately (Section 7). A snapshot therefore names the modules that are installed, and an operation against a module that is absent **MUST** fail with a distinguishable error rather than be treated as an empty module.

Which level of hash identifies a snapshot. The snapshot document is a transportable file, so its own identity is an OCI digest, not a fourth kind of hash (Section 6.8). It is made *of* Merkle roots and identified *by* an OCI digest, and the three-level separation exists to keep those straight.

It carries the trust root. The `trust_root` member holds the keys authorized to sign for this brain and the scopes each one holds. It lives here rather than in a module or a layer because it **MUST** reach every install, complete or partial (Section 8.4). The member **MAY** be absent, and absence and emptiness are equivalent: both mean a brain with no authorized keys, the zero-configuration case of Section 8.1. What neither means is deferral, and Section 8.6 states the consequence of starting without one.

Its timestamp decides nothing. The `created_at` member is descriptive. It is written by whoever built the snapshot, so a conforming implementation **MUST NOT** use it for any security or ordering decision. Ordering comes from the parent chain, which cannot be altered without changing every descendant (Section 8.5).

6.6 Indices

Indices are derived: they can be rebuilt from the blocks. They may travel inside the same module so that each consumer need not recompute them, but they never replace the content. Table 6 lists the indices and their roles.

Table 6: Indices, their typical queries, and their role.

Index	Typical query	Role
Hash map	Get exactly <code>sha256:ABC</code> .	Immediate physical resolution.
B ⁺ tree	Episodes between two dates; concept by ID.	Ordered keys and range scans.
Inverted	Blocks containing “Fourier”.	Lexical matching.
Vector	“Decompose a periodic function into sines”.	Semantic similarity.
Graph	Related concepts, evidence, dependencies.	Associative traversal.
Bitmap / facets	<code>semantic AND class=signals AND current</code> .	Efficient filter intersection.
Catalog	Everything filed under “Exams”.	Navigating evidence by axis.

Whether an index travels inside the module or is rebuilt on installation depends on whether it can be regenerated without a model. Structural indices (hash map, B⁺ tree, inverted, bitmap, and, when relations live on the blocks, the graph) are deterministic functions of the blocks and can be rebuilt cheaply by any client. The vector index is the exception: rebuilding it requires an embedding model, which a model-agnostic client does not carry. The vector index therefore travels with the module and records the embedding model and version that produced it, so that consumers can reproduce or compare it.

The same argument applies to a client that restarts: a structural index can be rebuilt whenever it is needed, while a client holding the vector index only in memory has lost it for good. An implementation must therefore persist the travelling index locally, alongside the module it belongs to, and must not republish a placeholder in its stead: an artifact whose layer claims a vector index and carries none is worse than one that omits the layer, because a consumer can detect the absence and cannot detect the emptiness.

A travelling index MUST be bound to the composition it was built over. Three requirements elsewhere in this document compose badly if this is left implicit: a travelling index cannot be rebuilt by a model-agnostic client, a drop rebuilds the module’s indices from the new composition (Section 12.3), and no operation removes a single entry from an index, because

indices are built and queried, never edited. An implementation that does the obvious thing therefore publishes, after a drop, an index that still contains the representation of the dropped block.

Queries stay correct, because a hit that is no longer a member fails membership verification and is filtered out (Section 11), and that is what makes it worse: nothing surfaces to be noticed, and the published artifact silently carries a learned representation of content that was deliberately excluded. A drop meant to destroy sensitive content leaks exactly what it was meant to remove, and text embeddings recover enough of their input for the concern to be a practical one [35]. Drop is the *ordinary* removal path (Section 12), so it is the common case that leaks.

Three rules close it. A travelling index **MUST** record the module root it was built over, so that staleness is detectable rather than invisible. An implementation **MUST NOT** publish an index built over a composition other than the one its module names: a model-bearing client rebuilds the index, and a client that cannot rebuild it **MUST** omit the layer rather than ship a stale one, which is the argument of the previous paragraph carried one step further. And a consumer that meets an index bound to a root other than the installed one **MUST** report it as stale and **MUST NOT** answer queries from it. Removing individual entries is permitted where an engine supports it and stays an implementation matter rather than a protocol operation, because deletion from an approximate nearest-neighbour structure is engine-specific and lossy. What the protocol requires is narrower and checkable: a published artifact **MUST NOT** claim an index it did not build over the composition it ships.

A fourth rule makes the binding hold against an adversarial registry rather than a careless publisher. When an index travels, the snapshot names the digest of the index payload alongside the embedding model (Section 6.5), which places the index inside the signed bytes, and a consumer **MUST** reject a travelling index whose payload does not hash to the digest its snapshot names. Without this the index would be the one component of the artifact covered by no signature, and whoever controls a registry or mirror could substitute it: nothing fabricated would surface, since every returned block still verifies, but what surfaces would be theirs to choose (Section 11.2).

The separation is also representational: a block carries meaning symbolically, in its text and its explicit relations to other blocks, while learned representations live in the vector index as derived, model-tagged views. Operating mathematically on concepts happens inside the external LLM, in its own representation space. This clarifies a naming coincidence: the *semantic* module denotes general, consolidated knowledge, as opposed to episodic memory, not an embedding-based representation.

6.7 The catalog: making canonical evidence navigable

A canonical module that holds five hundred sources is, to a consumer, five hundred hashes. The protocol says what each one is and proves that it is intact, and it says nothing at all about where any of it belongs. A brain published that way is a pile, and the party best placed to organize it, the one who assembled it, has no way to record the organization so that it travels with the artifact.

Section 11 already assumes otherwise: its request carries a class filter, which until now named nothing this document defines.

Where classification cannot live. Two placements are ruled out by constraints stated earlier.

It cannot live inside the canonical block. A canonical block carries only what its bytes determine, so that re-registering an identical original is a no-op (Section 5). A classification written into the payload would enter `block_id`, two parties filing the same PDF under different headings would compute different identities, and the evidence would fork every time someone changed their mind about where it goes.

It cannot live in provenance either, and the reason is practical. A processing task **MUST** NOT allow candidates in the canonical or provenance modules (Section 10), so a classification held there could never be proposed by a model. Filing a corpus of any size by hand is not a workable requirement.

Classification is consolidated knowledge. “This source is about Fourier analysis” is a claim derived from reading the source, which is what the semantic module holds and what the ingestion path produces. Placing it there means a model may propose it, the validation of Section 10.3 judges it, a drop removes it, and rederive rebuilds it. No new machinery is introduced.

Three kinds of block. A *scheme* declares an axis of classification and whether that axis is exclusive. A *class* is a node on one axis, carrying a label and the scheme it belongs to. A *placement* files one canonical block under one class, and cites that canonical block as its evidence, which makes the canonical cascade of Section 12.3 apply to it without a special rule: dropping a source removes its filings in the same commit, and nothing has to remember to clean up.

```
scheme      { "kind": "scheme", "scheme": "subject", "exclusive": false }
class       { "kind": "class",  "scheme": "subject", "label": "Fourier analysis" }
placement   { "kind": "relation", "evidence": ["sha256:EXAM_2024"],
               "relations": [{ "predicate": "classified_as",
                               "target": "sha256:CLASS_FOURIER" }] }
```

Hierarchy is a block, not a field. A class does not name its parent. The narrowing relation between two classes is its own block, naming both endpoints, and the asymmetry with a placement is deliberate: a placement takes its source from evidence and needs one edge, while a narrowing has two class endpoints and no evidence at all.

Recording the parent as a field on the class would put it inside that class’s `block_id`, so moving a node in the taxonomy would change the node’s identity, dangle every placement pointing at it, and invalidate everything filed beneath it. Reorganization is routine, so it must be cheap. This is the same reason supersession is a provenance edge and not a field on either block (Section 10).

One axis is never enough. A source on the Fourier transform is mathematics and is signal processing, and a past exam is both an exam and a document from 2024; forcing a single tree means choosing which of those questions a consumer may ask. Library classification answered this with facets long before content addressing existed: a document takes coordinates on several independent axes instead of one shelf position [46]. A brain therefore **MAY** declare several schemes, and a canonical block **MAY** hold placements on each of them. The broader and narrower vocabulary here is the one SKOS uses for the same purpose [33].

The catalog index. The classification blocks are the source of truth, in the sense Principle 1 requires, and the *catalog* is the derived index over them: what is filed under a class, which classes a source holds, and what the axes look like.

Unlike the vector index of Section 6.6, a catalog is a deterministic function of blocks a client already has, so any client rebuilds it without a model. It **MAY** nonetheless travel with the canonical module, bound to the composition it was built over under the same staleness rule, so that a consumer installing only canonical receives a navigable brain instead of a pile.

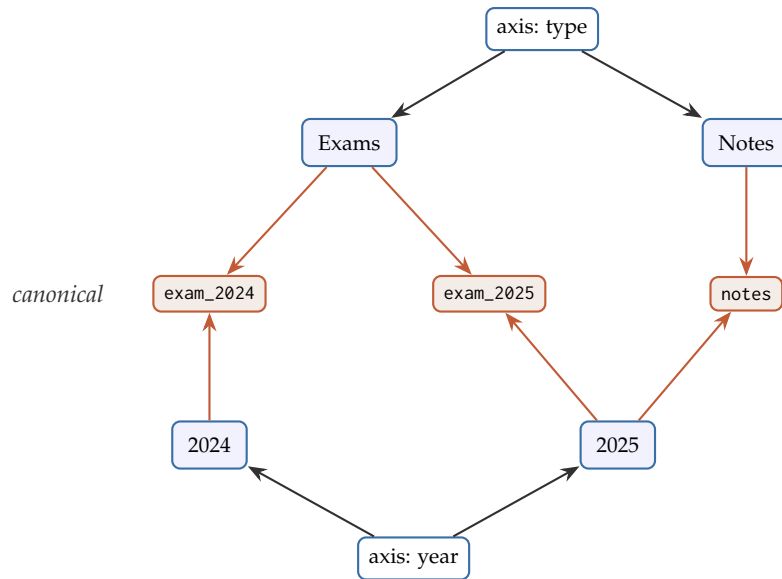


Figure 3: Two axes over the same three sources, grouping them differently: by type the sources split {exam_2024, exam_2025} against {notes}, and by year they split {exam_2024} against {exam_2025, notes}. Every accent edge is one placement block. No canonical block is copied and none of them changes, so adding, removing, or restructuring an axis touches only the semantic module and the canonical root a consumer already holds stays exactly as it was.

Normative rules. A canonical block **MUST NOT** carry classification. Reclassification **MUST NOT** change any canonical composition, so a consumer that already holds the evidence transfers nothing when a publisher reorganizes it. A class **MUST NOT** name its parent as a payload field. A placement **MUST** cite the canonical block it files as evidence. Where a scheme declares itself exclusive, a second placement for the same block on that axis **MUST** yield **CONTRADICTED** rather than silently coexist. And this document defines no scheme of its own: which axes exist, and what is on them, is a decision each brain records, in the same way it records a retention policy.

What this buys, and what it costs

The evidence is untouched, so reorganizing the whole library moves no bytes and invalidates no root. One source appears on as many axes as it deserves with no copy of it anywhere. A drop removes its filings through the cascade that already exists. Against that: classifying a large corpus for the first time is one model pass per source, and a consumer that installs no semantic module and receives no travelling catalog still sees the pile.

6.8 Three levels of hashes

A recurring source of confusion is that the same cryptographic algorithm identifies three different kinds of thing. A *block hash* identifies a logical unit of knowledge; a *Merkle root* identifies the logical composition of a module; an *OCI digest* identifies any transportable file, whether a published blob, a manifest, or one of the protocol's own documents.

The three use the same algorithm and do not identify the same kind of thing. The composition and snapshot documents of Sections 6.4 and 6.5 sit on this scale too, both made of lower-level identities while being identified at the third level. A composition document *contains* block hashes and *is* an OCI digest; a snapshot document *contains* Merkle roots and composition digests and *is* an OCI digest. Neither introduces a fourth level. The rule is that a level is determined by what a value identifies, not by what it is computed over.

6.9 Schema versions and forward compatibility

Block schemas evolve: a memory type acquires a field, and blocks written afterwards are not shaped like the ones written before. Because `schema_version` sits inside the envelope, and therefore inside `block_id`, this is not a cosmetic concern. Three rules keep an evolving brain readable.

Registered means registered with this protocol. “Registered” carries the identity layer, so it cannot stay implicit. The schema set is part of the specification: block schemas are published as a normative companion document, versioned with the protocol, and a schema is registered exactly when that companion carries it. Nothing looser survives contact with `block_id`: the version sits inside the identity, so a per-deployment registry would make identity comparable only within a deployment, and two parties holding the same knowledge would compute different names for it, the silent failure Section 6.2 exists to prevent, re-entering through the version field. Three consequences are normative. *Satisfies* is mechanical: a payload satisfies a schema exactly when it validates against it, with no coercion and no members the schema does not name. *Oldest* is by registration number: the companion assigns each memory type’s schemas consecutive versions, and the rule below selects the smallest-numbered schema the payload satisfies, which stays well-defined even when evolution is not monotonic, so a payload satisfying versions 1 and 3 but not 2 is written under 1. And an implementation claiming the writer role **MUST** implement the registered set for every memory type it writes, because a writer missing an older schema would version blocks the newer way and silently fork their identities.

A version is a statement, not a preference. A block **MUST** be written under the *oldest* registered schema its payload satisfies, and the proposer of a block does not get to choose. Writing under the newest schema instead would let a new schema silently re-version every block written afterwards, including blocks that use nothing it added, making an artifact unreadable to consumers who could have read its knowledge perfectly well. Oldest-that-fits inverts this: a brain stops being readable by an older client only at the point where it genuinely uses something that client has no schema for.

A version may also remove a field. Evolution is usually additive, but not always: provenance version 2 drops the producer member that version 1 requires, replacing it with the assisting parties of Section 5. This needs no new rule, because *satisfies* already admits no members a schema does not name. A payload carrying producer satisfies version 1 and not version 2; one carrying assisting parties satisfies version 2 and not version 1; one carrying both, or neither, satisfies no registered schema and **MUST** be refused. The two versions are disjoint rather than nested, and selection is therefore unambiguous without consulting anything outside the payload — which is the property oldest-that-fits exists to guarantee, and it survives a removal unchanged.

The consequence for a writer is the one that matters. A brain whose records name no assisting parties keeps writing version 1, byte for byte as before, and stays readable by every client that could read it yesterday. Only a record that actually names who assisted pays for version 2.

An unknown version **MUST fail loudly.** A block declaring a schema version a client does not implement was written by a newer implementation. The schema cannot be inferred from the bytes, so the client **MUST** refuse the block and say so, rather than parse what it recognizes and drop the rest.

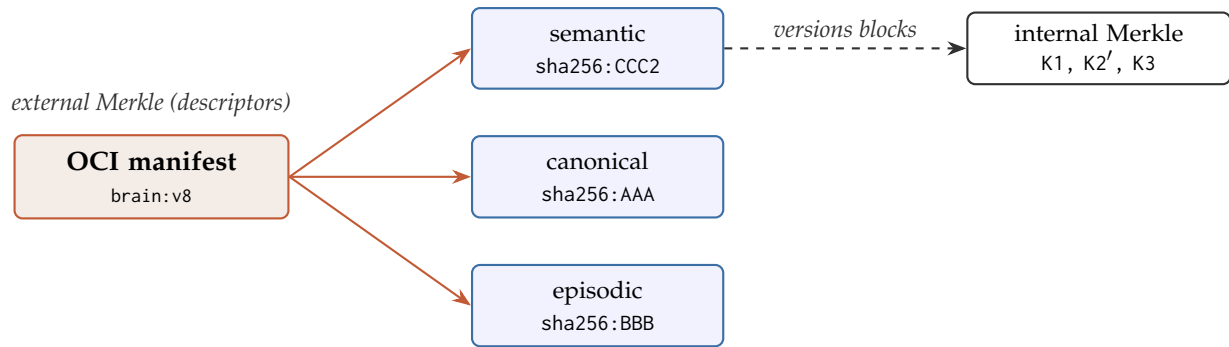


Figure 4: OCI distributes modules as separate blobs referenced by a small manifest; inside each module, an internal Merkle DAG versions the individual knowledge blocks.

The refusal must be possible before the download. Which schema versions a module holds is declared on the published artifact (Section 7). The information was always *in* the brain, since every envelope names its version, but reachable only by fetching and parsing blocks, which is to say after a download the consumer may not be able to use. Declaring it up front lets a client refuse a brain it cannot read before transferring a byte, and say why.

This is a different question from the protocol version, and the two **MUST** stay separate. The protocol version asks whether this is a Boltzmann artifact at all; the schema version asks whether this client has the schemas for the knowledge inside it. A brain using a schema a consumer lacks is still a brain, so that consumer **MUST** still be able to install the modules it *can* read, which a protocol-level refusal could not express.

7 Distribution with OCI Artifacts

OCI is a standard for content-addressed manifests, descriptors, and blobs. A compatible registry stores the blobs and allows referencing them by tag or digest. The specification organizes its components as an external Merkle DAG of descriptors [39, 40, 44]. Figure 4 shows how OCI distributes modules while the internal Merkle versions the blocks within each module.

7.1 Complete brain

A brain is published as an OCI Artifact whose *config blob is the snapshot document itself* (Section 6.5) and which carries one layer per module:

```

manifest artifactType: application/vnd.gaussia.boltzmann.brain.v1+json
config -> the snapshot document
layers
  canonical -> sha256:AAA   annotations: memory-type, merkle-root,
  episodic   -> sha256:BBB               merkle-layout, block-count
  semantic   -> sha256:CCC
  procedural  -> sha256:DDD
  provenance  -> sha256:EEE
  history     -> sha256:FFF   annotations: snapshot-count
  (vector index layers, when one travels)
  
```

The OCI manifest is small, and downloading it does not imply downloading any module. Making the snapshot the config blob means the same document is both the local state and the wire format, so publishing is a transfer rather than a conversion, and the trust root arrives with every install (Section 8.4). The config blob **MUST** be exactly the canonical serialization

of the snapshot document, and a consumer **MUST** reject an artifact whose config is not in canonical form; the same rule applies to composition documents wherever they travel. This is the decoding check of Section 6.1 applied one level up, and it is what keeps the digests that name a snapshot in three places, the OCI digest of the config, the entries of parents, and the digest a signature record covers, one value rather than three. Signatures are deliberately absent from this listing: they are published as separate artifacts referring to the manifest, for reasons Section 8.8 gives.

The history layer. A snapshot names its parents by digest, and several rules of Section 8 run by walking that chain: finding the trust-root revision in force, checking a quorum, computing the difference that decides required scopes. An artifact that carried only its head would name documents no consumer can resolve, leaving those rules checkable by nobody but the publisher. A complete brain therefore carries a *history layer*: the snapshot documents of the head's first-parent ancestry, in canonical bytes, together with the composition documents they reference. Compositions travel because required scopes are computed from differences between compositions; without a parent's compositions, every pulled commit is undecidable between an ingest and a canonical drop. Ancestor *blocks* do not travel. A publisher of a complete brain **MUST** include the ancestry back to the genesis snapshot; the cost is small, since a snapshot document is a few hundred bytes and compresses well against its near-identical siblings, and Section 8.9 states what a verifier reports when the chain it can resolve stops short.

Media types. Two clients that disagree on media types cannot pull each other's brains, so these are normative here (Table 7).

Table 7: Normative media types.

Role	Media type
Artifact type	application/vnd.gaussia.boltzmann.brain.v1+json
Config (snapshot)	application/vnd.gaussia.boltzmann.snapshot.v1+json
Config (projection)	application/vnd.gaussia.boltzmann.projection.v1+json
Module layer	application/vnd.gaussia.boltzmann.module.{type}.v1.tar+gzip
History layer	application/vnd.gaussia.boltzmann.history.v1+tar+gzip
Vector index layer	application/vnd.gaussia.boltzmann.index.vector.v1+bin
Signature record	application/vnd.gaussia.boltzmann.signature.v1+json

A module layer is a tar archive holding the module's composition document and its block bytes. An implementation **MUST NOT** repack a module whose composition is unchanged: it **MUST** reuse the previous layer digest, which is what keeps registry deduplication working across publishes. Packing **SHOULD** be deterministic within an implementation, so repeated packs of one composition yield one digest; byte-identical layers across implementations are deliberately not required, because the formats do not permit requiring them. RFC 1951 leaves block boundaries and match strategy to the compressor, and RFC 1952 leaves header fields implementation-dependent, so two conforming encoders legitimately produce different streams for the same input [13, 14]. The reference implementation packs with the PAX format, zeroed timestamps, sorted entries, and a pinned gzip level. Gzip is specified rather than a denser codec because it is universally available: a client that needed an extra dependency to read a published brain would be trading portability for a few percent.

Annotations. The manifest carries what a consumer needs to decide *before* downloading (Table 8). Annotations are unauthenticated hints: nothing signs a manifest, so every annotation

is writable by whoever controls the registry. They MAY drive the decision to transfer, which is their purpose, but a consumer MUST re-verify any value it acted on against the config blob and the downloaded bytes, and MUST NOT base a trust decision on an annotation alone. The trust-root comparison against a pin (Section 8.7) is a pre-download courtesy; the comparison that counts is against the trust root inside the config.

Table 8: Manifest and layer annotations, all prefixed `ai.gaussia.boltzmann`.

Annotation	What it declares
memory-type	Which module a layer holds. This is what a selective install resolves on.
merkle-root	The layer's internal Merkle root.
merkle-layout	The construction that produced the root; roots are comparable only between clients that agree here.
block-count	How many blocks the composition holds.
protocol-version	Which protocol version the artifact conforms to.
schema-versions	Which block schema versions each module holds (Section 6.9).
embedding-model	Model and version behind a vector index layer.
index-kind	Which kind of index a travelling index layer carries.
source-snapshot	The publisher's full snapshot this artifact was projected from.
trust-root	Digest of the trust root carried in the snapshot, so a consumer can compare it against a pin before transferring anything (Section 8.7).
index-root	The module root a travelling index layer was built over (Section 6.6).

The merkle-root annotation is deliberately distinct from the descriptor's own digest, and that distinction is the third level of Section 6.8 made operational: the digest identifies the transportable file, the root identifies the logical composition inside it. This is the annotation that closes the gap Section 4.3 identified in composing OCI with a retrieval stack.

Projections are views, not snapshots. Publishing a subset of modules does not mint a new snapshot. A document that omits modules is not a version of the brain and has no position in the chain: it would need parents that do not type-check, a signature over bytes that belong to no history, and scopes no difference can compute. The config blob of a subset artifact is instead a *projection document*, under its own media type (Table 7): it names the source snapshot by digest and carries, verbatim, the module references it retains. A projection is not signed and needs no scope, because its authority is the source's. A consumer verifies it by resolving the source snapshot, which travels in the artifact's history layer, verifying the source's signatures over the source's canonical bytes, and checking that every module reference in the projection is byte-identical to the corresponding entry of the source; a projection whose references are not a verbatim subset of its source MUST be rejected. Pinning, trust-root evaluation and ancestry all run against the source snapshot, so a projection is a safe first-contact object exactly to the extent its source is. The source-snapshot annotation stays a pre-download hint like every annotation; the binding that counts is the source digest inside the projection document, checked against the resolved source itself. And publishing back from a projection install is the partial-install reconciliation of Section 13.7, so nothing about a projection reads as a divergence.

7.2 Selective installation

To install only the episodic module, the client resolves the descriptor marked as episodic and downloads `sha256:BBB`. It does not need to download `canonical`, `semantic`, or `procedural`. It then opens the episodic Merkle DAG and its local indices.

Necessary condition

Selection is only efficient if each module is a separate OCI blob or artifact. If everything is mixed into a single file, the whole file must be downloaded.

7.3 Update granularity

The unit of transfer is the module. If only the semantic module changes, the new manifest reuses the digests of the remaining modules, and the consumer downloads only the new semantic blob. It downloads that blob whole, however small the change inside it: what is incremental at the block level is local storage and differencing, not transfer. Once the blob is on disk, comparing Merkle roots identifies exactly which logical blocks changed. This version claims no finer transfer granularity; per-block blobs would trade one download for thousands of registry round-trips, and modules are where selective installation lives.

Table 9: An update touches only the changed module's digest.

Version	Canonical	Episodic	Semantic	Procedural
v7	AAA	BBB	CCC	DDD
v8	AAA	BBB	CCC2	DDD

OCI contributes storage, registry authentication, transport, digest-based deduplication, and tags. Boltzmann Brain contributes the meaning of the modules, the blocks inside them, and their query contract.

7.4 Local state, atomicity, and divergence

Everything described so far is content-addressed and immutable. A working brain needs exactly one piece of mutable state: a pointer saying which snapshot is current. Isolating it to a single pointer makes a commit atomic, and it is the only place where write ordering matters. Named branches (Section 7.5) add a table of further pointers beside it, but a commit still moves only the current one.

Commit is a pointer move. A conforming implementation **MUST** write every blob a new snapshot references before moving the pointer, and **MUST** move the pointer last. Blobs written and never referenced are unreachable and will be reclaimed by pruning (Section 12); a pointer moved before its blobs exist would name a state that cannot be resolved. The asymmetry is why a commit can be interrupted without corrupting a brain.

Retained roots. Alongside the current snapshot, a brain keeps a set of retained snapshots: tagged releases, the head of every named branch, plus a bounded number of recent ones. Branch heads do not count against the bound. This set is what reachability is computed from when pruning reclaims blocks (Section 12), and gives “a retained root” a precise meaning.

Origin. A brain pulled from a registry **MUST** record where it came from: the repository reference, the tag, the snapshot the remote was on at pull time, and whether the install was partial. The recorded remote snapshot is what a later publish compares against. Content addressing alone would not protect a publisher here: overwriting a tag leaves every blob in place, but if no retained root names them, the work is gone in the only sense that matters.

Divergence is detected, not resolved. Before publishing, an implementation **MUST** compare the remote snapshot against the local chain. If the remote is an ancestor of the local snapshot, the publish fast-forwards. If it is not, the two brains have diverged, and the implementation **MUST** refuse the publish and report where the histories parted, rather than overwrite.

A publish is checked again after it lands. The comparison above and the write that follows it are two requests, and OCI offers no conditional write on a tag: a second publisher whose check ran in the gap overwrites the first, and both checks passed. The protocol cannot close that gap without a registry feature it does not assume, so it requires that the gap be detected. After moving a tag, an implementation **MUST** resolve it again and compare the snapshot it now names against the one it published. The same snapshot is success. An ancestor of the published snapshot means the registry has not yet served the write, and the implementation **SHOULD** retry the read before concluding anything. Any other snapshot means another publish replaced this one, and the implementation **MUST** report a *lost publish*, distinguishable from a divergence (Table 14). Nothing is lost locally: the published snapshot is still the local head, and the remedy is the one for divergence. Only the publisher that lost can observe the race; the one that won cannot, and neither can a client that skips the check.

The same comparison runs on pull. The comparison is not the publisher's alone. On pull, when the consumer already holds a head for this brain, an implementation **MUST** compare the served head against it. If the served head is a strict ancestor of the held one, the remote has walked backward, and the implementation **MUST** refuse to adopt it, or require an explicit override, and **MUST** report a *rollback*, distinguishable from a divergence and from any integrity failure (Table 14). The state this check needs is already mandated: *Origin* records the snapshot the remote was on at pull time, and the history layer resolves the ancestry. A consumer that pruned its local history below the served head cannot decide the question and **MAY** warn instead, the same undecidability the chain-truncated failure names. What the check buys is bounded in Section 14.3: it does not detect withholding from a newcomer; it makes walking a returning reader backward detectable with no party outside the artifact.

Detecting divergence is a reader's duty and belongs here; deciding what to do next is a separate question, and Section 13 answers it, including the partial install that cannot otherwise be published back (Section 13.7).

7.5 Branches

The divergence check makes a shared tag safe, not convenient. When several writers publish to one tag, every publish but the first is refused until its writer reconciles, so the tag serializes the whole team. A *branch* gives each line of work a tag of its own, and moves reconciliation to the moment a writer chooses to join one line into another.

A branch is a named pointer. A brain holds, beside the current-snapshot pointer, a table of *refs*: named pointers, each to a snapshot, each carrying its own *Origin*. One branch is current; its head is the current snapshot, so a commit moves the current pointer exactly as before and touches no ref. A brain with no ref table has one branch, the default branch, named *main*, and behaves exactly as this document described before branches existed. *checkout* records the current snapshot and *Origin* under the current branch's ref, then moves the current pointer to another branch's head and restores that branch's *Origin*. Like a commit, it **MUST** write the ref before it moves the pointer, so an interrupted checkout leaves both heads named. An implementation **MUST** refuse checkout while a reconciliation is open, because the open reconciliation is stated against the current head.

Branch heads are roots. Every branch head is a retained root (Section 7.4), and prune MUST treat it as one: the snapshot, its compositions, and every block they name stay reachable. A branch head that left the retained set when commits elsewhere pushed it past the bound would otherwise be reclaimed while its name still pointed at it. An implementation MUST also keep every branch head in the retained set itself, outside the bound, so that a client that knows nothing of refs and prunes from the retained set alone still keeps them.

A branch publishes to a tag. A branch name is one or more segments separated by /, each segment matching `[A-Za-z0-9][A-Za-z0-9_-]*`. The default branch publishes to the tag the deployment treats as current, conventionally latest. Every other branch publishes to the tag formed by the prefix `br.` followed by its name with each / replaced by `.`, so `ana/fix-typo` publishes to `br.ana.fix-typo`. Segments contain no `.`, so the mapping is reversible, and a tag that begins `br.` and decodes to a valid name names a branch to any client that lists the repository. An implementation MUST refuse a name whose tag would exceed the 128 characters OCI allows, and MUST refuse `main` as the name of a non-default branch. The prefix is fixed by this document, not by deployments, so that branches published by one client are branches to every other. Neither the branch nor the tag enters the manifest: a snapshot has the same manifest digest under every tag it is published to, which is what lets a branch be created on the registry by publishing a snapshot it already holds. `push` and `pull` default to the current branch's tag, and an explicit tag overrides the default without renaming the branch. The divergence check (Section 7.4) runs per tag, so two writers on different branches never refuse each other.

Pulling another branch's tag. A pull whose tag names a branch other than the current one installs into that branch, creating its ref if the brain has none, and makes it current. It does not overwrite the current branch: the current head is recorded under its ref first, exactly as checkout records it.

Joining branches. Joining a branch into the current one needs no new mechanism. If the current head is an ancestor of the other branch's head, the join fast-forwards: the current pointer moves to that head and no snapshot is written. If the other head is already reachable from the current one, there is nothing to join. Otherwise the two heads have diverged, and the join is a reconciliation (Section 13) whose incoming history is the other branch's head, already held locally, recorded by whichever strategy the writer chooses.

Deleting a branch. Deleting a branch removes its ref. An implementation MUST refuse to delete the current branch, and MUST refuse, absent an explicit override, to delete a branch whose head is reachable from no other branch and was never published, because the ref is then the only name for that work. Deleting a published branch from the registry is out of scope: OCI deletes manifests by digest, and a branch's manifest may be the same manifest another tag names.

8 Authenticity and Trust

Everything specified so far establishes that a brain is *intact*. Nothing specified so far establishes *who assembled it*. Those are two different claims, and this section separates them, identifies the single assertion the hash structure cannot make on its own, and fixes the smallest structure that can carry it.

8.1 What the hash structure already proves, and the one thing it cannot

The protocol stacks four commitments, each one pinning the level below it. A `block_id` commits to a block's bytes, including the memory type, schema version, and serialization named in its envelope (Section 6.1). A Merkle root commits to the exact set of block identifiers in a module (Section 6.3). A composition document makes that set *resolvable* rather than merely verifiable, and is itself content-addressed (Section 6.4). A snapshot commits to one root per module, and to its parents by digest (Section 6.5).

Because the parents are named by digest, the chain of snapshots is itself hash-linked. Altering anything anywhere, a byte in a source, a block's membership in a module, an entry in an old snapshot, changes an identifier, which changes every value above and after it.

Two consequences follow, in opposite directions.

Integrity is already total and transitive. Any consumer, offline, with no configuration and no trust in anyone, can recompute the whole structure and confirm that a brain is exactly the brain those identifiers describe. Nothing needs to be added for that, and nothing in this section may be allowed to make it conditional.

Authorship is unreachable from inside. Every check above is a check of internal consistency, and internal consistency is cheap to manufacture. The attack requires no cleverness: copy a brain, add a fabricated semantic block, recompute the affected roots, publish a consistent snapshot under someone else's name. The structure proves that a brain is coherent; it cannot distinguish a coherent brain from a *particular party's* coherent brain.

This is worse than a missing feature, because without it Principle 8 does not hold: provenance is made of blocks like everything else, and the actor it records is a *declared identifier* rather than an authenticated identity, so whoever can write to a brain can forge its audit trail. A signature is what gives that actor field meaning.

So what is missing is not more hashing. It is a single assertion, by a party, that the hashes cannot make on their own.

8.2 Where a signature attaches

The hash structure determines this almost completely: **one signature over the snapshot covers everything.**

Signing a snapshot commits to the module roots it names. Each root commits to that module's entire block set. Each block commits to its own bytes. The parent digests commit to the whole prior history, since changing any ancestor would change its digest and break the chain leading to the signed value. A conforming implementation **MUST** therefore attach signatures to snapshots, and **MUST NOT** require a signature over any other object to establish authorship. Signing the head is sufficient for content: the parent digests already commit to every ancestor's composition, so re-signing history adds nothing to integrity, and signing every historical snapshot is **OPTIONAL**. Authority is the exception. A trust-root revision is legitimate only through the govern signatures that approved it (Section 8.4), so the signature records covering every trust-root revision in the ancestry **MUST** travel with the brain, or the quorum rule becomes uncheckable by anyone downstream.

Because the snapshot names each module's Merkle layout and each block envelope names its serialization, a signature commits to *how* the values were computed and not merely to the values, so a consumer cannot be steered into recomputing under a different construction.

Blocks MUST NOT carry the signer. Block identity must depend on the hashed bytes alone (Section 5), so a signer entering the hashed content would make two parties ingesting the same

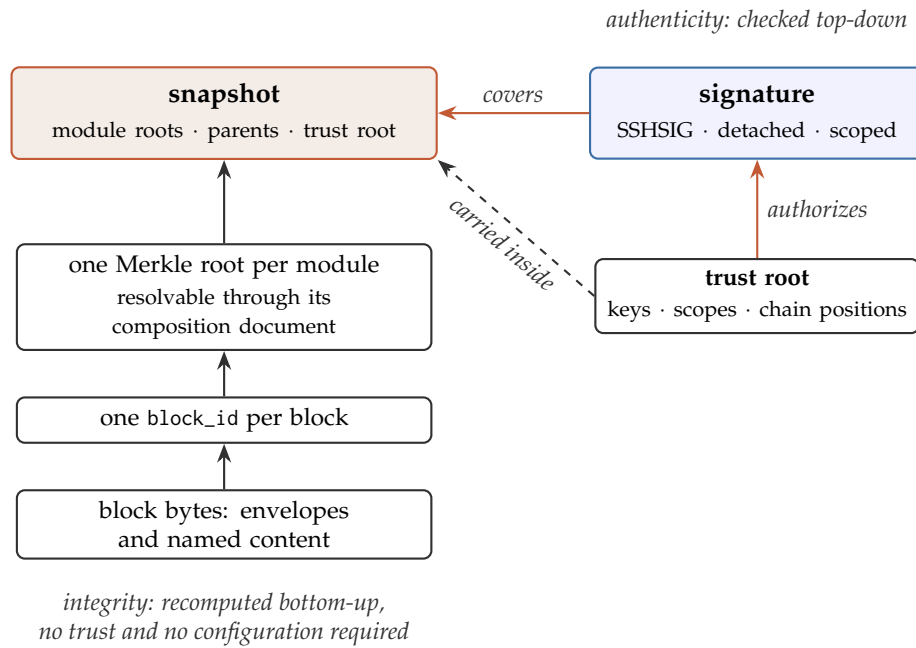


Figure 5: The two verifications and where they meet. The left column is recomputed from the bytes upward and needs no trust in anyone; the snapshot is the only place a signature attaches, because it is the only document that commits to every module root at once. The trust root travels inside the snapshot and therefore inside the signed bytes.

PDF produce different identifiers and collapse structural sharing. Blocks stay anonymous and immutable; snapshots carry attribution.

Signing module roots individually is not a substitute. Five independent signatures are five independent claims, and nothing binds them into a single state. An attacker who holds five separately signed roots can combine today's semantic root with yesterday's canonical root, the one that still contained a source since dropped, and both signatures verify. The consumer installs a brain that never existed, holding evidence that was deliberately excluded, with every signature checking out. Only a signature over the document that *combines* the roots prevents this, which is precisely what a snapshot is. A conforming implementation **MUST NOT** accept per-root signatures in place of a signature over a snapshot.

8.3 The signing mechanism

Signatures are made with the SSH signature format, SSHSIG [42], the same mechanism Git uses when configured with `gpg.format=ssh`. There is no PKI and no certificate authority: contributors already hold SSH keys, and the forges already publish them. Ed25519 [21] is the RECOMMENDED key type.

A signature **MUST** be produced over the canonical bytes of the snapshot document (Section 6.2) under the namespace `boltzmann.snapshot.v1`. The namespace is not decoration: it is what prevents cross-protocol replay, so that a signature a contributor produced for a Git commit cannot be presented as a Boltzmann signature, and a later version of this document can introduce a new namespace without invalidating signatures made under this one. A conforming implementation **MUST** reject a signature made under any other namespace. The namespace separates protocols, not brains: a signature binds to bytes, and what separates two brains is their genesis (Section 8.7).

The parameters are pinned. SSHSIG admits sha256 and sha512 in its `hash_algorithm` field and expresses no preference, and the field sits inside the signed blob, so two implementations that chose differently would produce non-interoperable signatures over identical knowledge, and the signature vectors of Section 15 would not be reproducible. This protocol pins it: a signature **MUST** be produced with `hash_algorithm sha512`, and a verifier **MUST** reject a signature naming any other value. Key types are bounded from below as well: SSHSIG itself already excludes legacy `ssh-rsa` signatures over SHA-1, and a verifier **MUST** additionally reject `ssh-dss` keys and RSA keys shorter than 3072 bits.

What valid means for Ed25519 is pinned too. Naming the key type does not settle what *valid* means. RFC 8032 sanctions two acceptance criteria for the same signature, calling the stricter check “sufficient, but not required” [21], and measurement across sixteen deployed libraries finds them disagreeing in both directions on the same inputs, most of them accepting encodings the RFC mandates rejecting [7, 12]. Worse for this section’s purpose, the common lenient behaviour is not strongly binding: a signature that verifies under more than one public key is a signature its signer can later disown, and Section 8.1 rests on the signature being what makes the provenance actor field mean anything. A conforming verifier therefore **MUST** check the cofactorless equation $[S]B = R + [k]A$, which is what nearly every deployed library computes and batch verification, the one reason to prefer the cofactored form, is not used here; **MUST** reject a signature whose scalar S is not less than the group order; **MUST** reject non-canonical encodings of R or of the public key; and **MUST** reject a public key of small order. Each is an inexpensive check on bytes, performable before any library call, and together they make one verdict reachable per vector.

Non-goal: key management

The private key never enters the protocol. It stays where SSH keys already live: an agent, a hardware token, or a key-management service. The brain does not manage keys, for the same reason it does not embed a language model. A conforming implementation **MUST NOT** require a private key to be stored inside a brain, and **MUST NOT** define a format for one.

Signatures are detached. This is deliberately unlike Git, which embeds the signature inside the commit object and therefore changes the commit’s identity when it is re-signed. Keeping signatures outside the document they cover buys three things:

- Snapshot identity does not depend on whether, or by whom, a snapshot was signed. The converse constraint follows from labels living inside the document: re-labelling produces a new snapshot identity, and existing signatures do not carry over.
- A snapshot committed locally without a signature becomes signed at publication *without changing identity*, which matters because Section 7.4 already separates local state from publication.
- Several signatures can cover one snapshot. This is the only way to express a quorum, and it is the reason the choice is not merely stylistic.

A signature record names what it covers, under which namespace, by which key, and which scopes it claims:

```
{
  "boltzmann": 1,
  "snapshot": "sha256:SNAPSHOT_DIGEST",
```

```

"namespace": "boltzmann.snapshot.v1",
"key":       "SHA256:5tS7wqf2K...",
"scopes":    ["commit"],
"signature": "-----BEGIN SSH SIGNATURE----- ..."
}

```

The key field is the signing key's SSH fingerprint, and it is an index rather than an authority: SSHSIG carries the public key inside the signature blob, so a verifier MUST check that the key inside the blob matches an authorized entry of the trust root, and MUST NOT trust the named fingerprint on its own. A record whose fingerprint and embedded key disagree MUST be rejected.

8.4 The trust root

A signature answers *who*. It does not answer *and why should that matter*. Answering the second requires knowing which keys are authorized for this brain, and a consumer pulling from a mirror, a cache, or a tarball MUST be able to learn that without reaching a server, or the offline property of Section 8.1 is lost the moment authenticity is introduced.

The authorized keys therefore travel *inside* the brain, as a *trust root* carried in the snapshot document under the key `trust_root`:

```

{
  "boltzmann": 1,
  "revision": 3,
  "namespace": "boltzmann.snapshot.v1",
  "govern_quorum": 2,
  "keys": [
    { "key": "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "subject": "alex@example.org",
      "scopes": ["ingest", "commit", "drop:canonical", "govern"],
      "since": 1 },
    { "key": "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "subject": "example.com/juan",
      "scopes": ["ingest", "commit", "govern"],
      "since": 1 },
    { "key": "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "scopes": ["propose"],
      "since": 2 },
    { "key": "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "scopes": ["commit"],
      "since": 1,
      "retired_from": 3 },
    { "key": "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "scopes": ["commit"],
      "since": 1,
      "compromised_from": "sha256:SNAPSHOT_S41" }
  ]
}

```

Its identity is the digest of its canonical serialization, and that digest, rather than any individual key, is what a consumer pins (Section 8.7).

What a key's subject does and does not claim. An entry MAY carry a subject: an actor identifier (Section 5) naming who holds the key. It is what finally connects the two halves of Section 5's promise — that a signature is what turns a declared actor into an authenticated identity — because without it a snapshot's provenance names a person and its signature names a fingerprint, with nothing asserting the two are the same and no way for a verifier to ask.

The claim it makes is deliberately narrow. A subject is asserted by this brain's governance and nowhere else: changing one changes the trust root, which is a revision, which requires a quorum evaluated against the previous revision (Section 8.5). It is therefore as trustworthy as the quorum that recorded it and no more. It is emphatically not a certificate. The grounds for believing a key belongs to a particular person in the world remain outside the protocol, exactly as Section 8.10 says; what a subject adds is that once those grounds exist, the conclusion can be written down where a verifier will read it, instead of living in a maintainer's memory.

The member MAY be absent, and absence is the ordinary case for a brain that has never needed it. An absent subject MUST be omitted rather than serialized as null, so that a trust root that names no subjects has exactly the digest it had before this member existed, and every pin taken against it continues to hold.

Two forms of position, and why they are not interchangeable. `since` and `retired_from` name a *revision number*; `compromised_from` names a *snapshot digest*. The asymmetry is a construction constraint rather than a preference. A revision travels inside the snapshot that introduces it, so naming that snapshot by digest from within the trust root would be circular. A revision number breaks the cycle and gives up nothing, because a revision corresponds to exactly one snapshot and the position is recoverable by walking parents. A `compromised_from` always names a position already published, so its digest is known and no cycle arises; the finer granularity is necessary because a revision may be in force across many snapshots, and a key compromised mid-span cannot be described by a revision boundary.

One consequence is normative, because it removes an obvious lie. A revision that admitted a key is itself in the chain, so `since` is *confirmable* rather than merely asserted: a verifier walking revisions sees when a key first appeared. A trust root claiming a `since` earlier than the chain supports MUST be rejected, which stops a key that has just added itself from also claiming to have been authorized from the beginning.

Why it is not a sixth module. Modules are selectively installable (Section 7.2), so a consumer who installed only the semantic module would not receive the document needed to verify anything at all. The key list has to reach every install, complete or partial, which places it at the brain level. Carrying it in the snapshot, the config blob of a published artifact, makes omitting it structurally impossible, and puts the list inside the signed bytes, so a signature can never be evaluated against a key list the signer did not commit to.

Scopes are derived from operations, not invented. A scope names something the protocol already defines rather than a generic role, so that a consumer can check a claim mechanically (Table 10). This operationalizes Principle 2: "dropping canonical evidence is allowed only under explicit policy" today names a policy that a consumer has no way to confirm was honoured, and `drop:canonical` is what turns it into a checkable statement about who was permitted to do it.

An ordinary drop needs only `commit` while a canonical drop needs its own scope, and the asymmetry is the one Principle 2 already draws: an ordinary drop is recoverable by a later `commit` that re-adds the block, whereas excluding canonical evidence forfeits re-derivation from that source.

A verifier MUST compute the required scope set from the difference between the snapshot and its first parent, or, for a snapshot that has no parents, from its difference against the empty

Table 10: Signing scopes, and how a verifier decides which one a snapshot required. The requirement is computed from the difference between a snapshot and its first parent, never taken from what the signature claims.

Scope	Authorizes	Required when, relative to the first parent
ingest	Preserving a source as canonical evidence.	The canonical composition gained blocks.
commit	Adding derived blocks; ordinary drops, supersession, and demotion.	A non-canonical composition changed.
drop:canonical	Excluding canonical evidence, with its cascade.	The canonical composition lost blocks.
redact	Destroying bytes a retained root still names.	A module's tombstones list grew.
govern	Revising the trust root.	The trust root digest changed.
propose	Offering a snapshot for someone else's consideration.	Never sufficient for a published head (Section 13.6).

brain (Section 8.6), and MUST reject a signature whose key does not hold every scope in that set, even if the signature claims fewer. The scopes field of a signature record is therefore a statement of intent that aids diagnosis, never the basis of the decision: a scope states what the snapshot did, not what the signer claims.

The trust root is not access control

Nothing can stop someone modifying a copy of a brain they already hold. The brain is data on their disk, and a field claiming otherwise would be honoured only by clients that chose to honour it. The trust root is a rule the *reader* applies. It does not prevent writing; it prevents *impersonation*. Preventing writes to a particular repository is the registry's job, and the two mechanisms defend against different attackers: registry permissions protect a repository, signatures protect an identity everywhere, including after the bytes have left that repository.

Changing the trust root requires a quorum. If the key list were just another document, whoever can sign could add themselves to it and the structure would authorize itself. A *trust-root revision* is therefore a snapshot that has a first parent and whose trust_root digest differs from that parent's, and it MUST satisfy two conditions: it MUST be covered by at least govern_quorum valid signatures from distinct keys, and each of those keys MUST hold the govern scope in the trust root as it stood *before* the change. Both halves matter, and evaluating against the previous revision is the half that is easy to lose. This is the root-rotation rule of The Update Framework [52], which is the studied answer to exactly this problem.

A trust-root revision MUST NOT change anything else: its module roots MUST equal its first parent's. Governance is thereby a separate, individually auditable event in the chain rather than something folded into a commit, and a consumer can enumerate every change of authority a brain has ever undergone by walking the parents and comparing one digest.

A govern_quorum of 1 already stops a stranger, who holds no listed key at all. A quorum of 2 or more protects a brain against a single stolen legitimate key, which is the case worth designing for. The quorum needs headroom, though, and the reason is the failure the rule itself creates: when the quorum equals the number of govern holders, one compromised or simply lost key freezes governance permanently, since neither the attacker nor the remaining honest holders

can assemble the signatures to publish a `compromised_from` record or admit a replacement, while the attacker keeps signing within the stolen key's scopes. A deployment SHOULD keep more govern holders than the quorum requires, and an implementation SHOULD warn when creating or revising a trust root in which the two are equal. There is no recovery path inside the protocol: re-anchoring would be exactly the self-assertion the quorum rule exists to forbid, so the deadlock is a limitation this design accepts, and Section 16 lists it as open.

8.5 Retirement and revocation, without clocks

Removing a key and repudiating a key look similar and must behave oppositely.

When someone leaves a project, everything they signed while authorized SHOULD remain valid. If removal were retroactive, a routine departure would invalidate every snapshot that person ever signed, and large stretches of history would stop verifying for an administrative reason.

When a key is compromised, the opposite is required: everything it signed from the point of compromise onward MUST stop being trusted, even though it was signed while the key was still listed.

Separating the two requires knowing *when* a snapshot was signed, and the obvious approach fails. A timestamp inside a signature is written by the signer, so an attacker holding a stolen key can backdate freely; Git has the same hole. The `created_at` field of a snapshot is no better, being written by whoever built it.

The deployed alternative, and what refusing it costs. Rejecting self-asserted timestamps does not settle the question, because the deployed alternative is time nobody in the transaction asserts. C2PA, the closest comparable standard, adjudicates exactly this problem, signature validity after key expiry or revocation, with trusted third parties: every claim is timestamped by an RFC 3161 authority, signing certificates chain to a curated trust list, revocation is checked through OCSP, and validators are instructed to use the attested time, not the current time, when deciding whether the credential was valid, precisely so that claims can be validated indefinitely [1, 8]. That buys post-expiry adjudication forever, and its price is the standing third party: a timestamp authority at signing time, revocation infrastructure at validation time, and a consumer that can reach both. This design refuses the third party, because the offline property of Section 8.1 is the thing it will not trade, and takes the other side of the same bargain: positional validity needs nobody outside the artifact, and in exchange there is no external authority to appeal to when governance itself fails.

The hash structure supplies the missing ordering. Snapshots are chained by digest, so a snapshot's position in the chain cannot be forged: inserting or moving one would change every descendant's parent digest, and those are already published. Validity is therefore expressed *positionally* rather than temporally, and the question a verifier asks is:

Was this key authorized, with the scope this change required, in the trust root in force at this position in the chain?

Because trust-root revisions are themselves snapshots in that chain, a verifier walks back from a snapshot to find the revision in force and answers the question with no clock involved. Two record forms express the two intents, and a conforming implementation MUST distinguish them:

retired_from The revision from which the key is no longer authorized. Signatures at earlier positions remain valid. Retirement can never invalidate a signature that was valid before it, which makes an ordinary departure harmless.

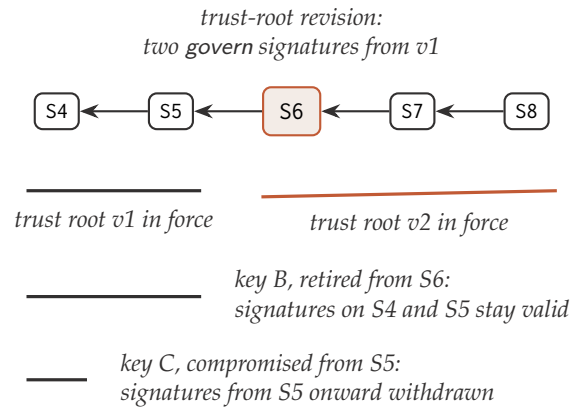


Figure 6: Validity is positional. A trust-root revision is an ordinary snapshot in the chain that changes the key list and nothing else, so the revision in force at any position is found by walking parents. Retirement closes an interval without disturbing what came before it; a compromise record closes one at a position that may precede the revision recording it, and is the only construct that withdraws a signature already accepted.

compromised_from A *snapshot* from which the key’s signatures are no longer trusted. It MAY precede the revision that records it, and it need not fall on a revision boundary, because compromise is discovered after the fact and in the middle of a revision’s span. This is the only construct in this document that invalidates a previously valid signature, and a verifier MUST report it as such rather than as an ordinary authorization failure.

In operation, “signed by a retired key, still valid” and “signed by a compromised key, no longer trusted” are different facts about the same snapshot, and collapsing them either breaks history or hides an attack.

Hash-linking was introduced in Section 6.5 for versioning, and supplies the unforgeable ordering that revocation needs. The closest precedents are clock-free in the same way. Keybase sigchains decide a statement’s validity by its position, and “old links remain valid even if their signing keys are revoked later”, which is this section’s retirement semantics [22]; OpenSSH key revocation lists revoke by serial, identifier or hash, with no validity window [43]; and The Update Framework orders trust-root rotations by monotonic version number, each new root signed by thresholds of both its predecessor and itself, which is the quorum rule of Section 8.4 under another name [59]. Certificate Transparency reaches for the ordering property through the same tree structure, at ecosystem scale and with a log operator this design does not have [29].

8.6 Creating a brain: the genesis snapshot

Every rule stated so far is expressed against a snapshot’s first parent, and one snapshot in every brain has none. The `init` operation produces it, and it is the origin of every authority the brain will ever have.

A *genesis snapshot* is a snapshot whose parents list is empty. A brain MUST have exactly one, and every other snapshot MUST be reachable from it by following parents. Its module compositions are usually empty, and an empty composition has a well-defined root (Section 6.3), so a brain with no knowledge in it is still a brain that verifies.

Three rules follow, each closing a gap the parent-relative phrasing leaves open.

Its difference is taken against the empty brain. A verifier computes the scopes a snapshot required from its difference with its first parent (Section 8.4). For a genesis snapshot that difference is taken against a brain with no modules and no trust root, so every scope its content

implies is required at once: govern because it establishes a trust root, and ingest or commit if it also carries blocks. A single self-signing key must therefore hold all of them, which is worth knowing before creating a brain with a narrowly scoped key.

The quorum rule does not govern it, and this is the only exception. A trust-root revision must be signed by keys authorized in the previous revision (Section 8.4), and a genesis snapshot has no previous revision to draw on. It is therefore not a revision, and a verifier **MUST NOT** reject it for failing the quorum rule. This is the single point in the protocol where authority is asserted rather than derived, and Section 8.7 is about what to do with that fact.

It SHOULD nonetheless satisfy its own declared quorum. A genesis snapshot **SHOULD** carry at least `govern_quorum` valid signatures from distinct keys listed in its own trust root. This proves nothing against an attacker, who can satisfy it just as easily. It is required for coherence: a brain that declares a quorum of two and whose founding act carried one signature is stating a rule it has already departed from. An implementation **MAY** warn when a genesis snapshot carries fewer signatures than the quorum it declares.

A genesis snapshot **MAY** be unsigned, and a brain whose genesis is unsigned is a brain with no authorship from the outset. That is the zero-configuration case of Section 8.1, and it remains fully verifiable for integrity. What it cannot later acquire is a signed history: signatures attach to snapshots, and the snapshots that were committed unsigned stay unsigned. The stronger consequence runs forward. A trust-root revision draws its quorum from the previous revision (Section 8.4), and a chain whose genesis carried no trust root has no revision to draw from, ever: governance cannot be adopted later, and a snapshot that introduces a trust root onto such a chain **MUST** be reported as a quorum failure, because no quorum could have approved it and every authority downstream of it is asserted rather than granted. Acquiring governed authorship means creating a new brain whose genesis carries the trust root. An implementation **SHOULD** warn at `init` when no trust root is supplied: trying the protocol unsigned and adopting signatures once the project matters is the natural path, and it is exactly the path this closes.

8.7 Bootstrap: the one thing that comes from outside

The quorum rule cannot apply to the first trust root, because there is no previous revision to draw a quorum from. The first list is self-signed, and read from inside the artifact it proves nothing: it asserts that a key is authorized, and the entity asserting it is that key. An attacker can construct an identical-looking object.

This is not a defect specific to this design. Trust cannot be manufactured from inside a system; at some point something must arrive from elsewhere. What a design can do is reduce it to a single decision, taken once. Two mechanisms are specified, and an implementation **MUST** support both:

Trust on first use, then pin. On first pull the consumer records the digest of the trust root and **MUST** warn loudly, and by default refuse, on any later change that does not follow the quorum rule of Section 8.4. This is the `known_hosts` model of SSH [62]. It does not prove the first contact was legitimate, but it converts an ongoing exposure into a single moment of exposure.

Pin out of band. The digest is published through an independent channel, a project site, a repository README, or DNS, and compared once by hand. Compromising the distribution then requires compromising two independent things.

Trust on first use is the RECOMMENDED default, with out-of-band pinning available to anyone who wants a stronger anchor. The residual limitation: a consumer who has never seen a given brain has no basis on which to reject an impostor's, and no arrangement of data inside the artifact can change that. Every comparable system carries this problem, and transparency logs are another answer to it [29, 38].

What a pin names: brain identity. A pin needs a key to file itself under, and the protocol defines one: the identity of a brain is the digest of its genesis snapshot. Nothing else is stable enough to carry it. Repository and tag are locators a brain outlives (Section 7.4 records them as local state), labels are re-assignable, and the trust root rotates; the genesis digest never changes, every snapshot chains back to it, and two heads belong to the same brain exactly when they resolve to the same genesis. A consumer's pin table **MUST** be keyed by genesis digest, and a pin record **SHOULD** retain that digest alongside the pinned revision, so a brain that moved repositories is recognized rather than re-anchored as a stranger. One consequence is stated rather than hidden: under content addressing, byte-identical geneses are one identity by definition, so a signature over a genesis binds to every deployment reusing those exact bytes, and the protocol-wide SSHSIG namespace does not separate them. A deployment that wants a distinct identity makes its genesis distinct, which `created_at` already provides unless bytes are copied deliberately, and which labels can make explicit.

Pinning also closes the last impersonation route (Section 8.11).

8.8 How a signed brain is published

The trust root has a home already: it is inside the snapshot, and the snapshot is the config blob (Section 7). Signatures need a different answer, because they must be able to accumulate.

A signature cannot simply be a layer. Adding a second signature to satisfy a quorum would change the manifest, and therefore the brain's digest, so a brain would change identity because someone countersigned it, with no knowledge having changed. That is what detaching signatures was meant to avoid.

The OCI Referrers API exists for this [41]. A signature is published as a separate artifact whose subject is the brain's manifest, which is how signature attestations already work [38, 58], so signatures accumulate *around* an artifact without touching it. A conforming implementation **MUST** publish signatures this way, so that registry tooling can discover them without implementing this protocol. The Referrers API is not universally present: the OCI distribution specification requires registries to support only the pull category, and instructs clients that meet a registry without the API to fall back to an image index at a tag derived from the subject digest, the referrers tag schema [41]. An implementation **MUST** implement that fallback, for publication and for discovery; a publisher that skipped it on such a registry would appear to succeed and leave the brain discoverable as unsigned.

But referrers exist only inside an OCI registry, and a signature that lives only there does not survive export to a tarball, a repository, or a disk. So both are required: signature records are part of the brain format and **MUST** travel with an exported brain, alongside the snapshot, and publication additionally mirrors them into referrers.

8.9 How verification runs

A consumer performs two independent checks and **MUST** report them separately rather than as one boolean.

1. **Integrity, bottom-up.** Recompute block identifiers from stored bytes, recompute each module's root from its composition, and confirm the snapshot names those roots. This requires nothing but the bytes, and its result **MUST NOT** depend on any trust configuration.

Table 11: Where each authenticity object lives, and why it cannot live elsewhere.

Object	Where it lives	Why not elsewhere
Trust root	Inside the snapshot document, which is the config blob of a published brain.	As a module or a layer it could be omitted by a selective install, leaving a consumer with nothing to verify against.
Signatures	Beside the snapshot in an export; mirrored as OCI referrer artifacts on publication.	As a layer, countersigning would change the brain's digest; as referrers only, they would not survive leaving a registry.
Pinned digest	In consumer-side state, never in the artifact.	A pin an artifact could supply would be a pin the attacker supplies.

2. **Authenticity, top-down.** Verify a signature over the snapshot's canonical bytes under the protocol namespace; confirm the key inside the signature is present in the trust root in force at that position in the chain; confirm the key holds every scope the snapshot's difference from its first parent required; confirm, where a pin exists, that the revision in force either matches the pin or descends from it through revisions each of which satisfied its predecessor's quorum; and confirm that the key is neither retired nor compromised at that position.

Attribution is reported, never enforced. Where a trust root declares subjects (Section 8.4), a verifier SHOULD additionally compare the actors named by the provenance records a snapshot adds against its first parent with the subjects of the keys that signed it, and report each actor as *verified* when a signing key vouches for it and *asserted* when none does. Assisting parties are never subject to this comparison: nothing expects a model to have signed anything.

The comparison MUST NOT block. A snapshot legitimately carries records whose actors never signed it — every merge does, since reconciliation brings another party's records into a history the local key signs (Section 13.3) — so a verifier that refused an unvouched actor would refuse the ordinary operation of Section 13. What it must not do is pass silently: an actor nobody vouches for is a declared name, which is what it was before subjects existed, and saying so is the whole value of having asked.

Authority is a property of the chain. The authenticity check does not stop at the head. A verifier MUST NOT report a snapshot as authorized unless the first-parent chain resolves from the head back to its anchor, which is the pinned trust root where one exists and the genesis on first contact; every trust-root revision along that chain carries the govern quorum its predecessor required; and every ancestor's difference from its own first parent was covered by the scopes its signer held. The last clause makes the forward propagation of rejection in Section 8.10 normative rather than narrative: a head whose ancestry contains an unauthorized change is unauthorized, however clean the head itself looks, and without this rule whoever merges could launder a privileged change by measuring their diff against a branch that had already made it. When the chain cannot be resolved that far, because history was withheld or a mirror truncated it, the verifier MUST report *chain truncated* (Table 14): integrity is intact, and authority above the deepest resolvable snapshot is undecidable. A partial install does not weaken the rule, because the history layer carries documents rather than modules, so ancestry resolves regardless of which modules are held.

What the pin means after a rotation. A pin is an anchor, not the current value. After a legitimate rotation the trust root in force no longer equals the pinned digest by construction, so digest equality would fail every valid rotation ever made; what verification checks is reachability, exactly as Case 1 answers it in Section 8.10: revision 2 is trustworthy because the chain introducing it satisfied revision 1's quorum, and revision 1 is the pin. Two consequences are normative. A consumer MAY advance its pin to the newest validated revision, which bounds re-validation to the suffix it has not yet walked. Advanced or not, a consumer MUST refuse a head whose revision in force is an ancestor of a revision it has already validated, reported as a regression rather than as an unauthorized key, because walking governance backward is how a stolen, once-valid key becomes current again. Removing or replacing a pin is an operator action outside the protocol, exactly as deleting a `known_hosts` entry is, and no data inside an artifact may cause it.

Keeping them distinct matters in practice. A consumer with no trust anchor can still verify integrity completely, which is the zero-configuration case that must keep working. And a consumer that reports a single verified flag cannot express "intact, and signed by an authorized key" as distinct from "intact, provenance unknown", which is the distinction Section 11 carries into the Evidence Bundle.

Verification policy is configuration. As with query planning and retention, the protocol fixes the checks and the reporting, not the deployment's appetite for risk. A consumer-side *verification policy* states whether an unsigned brain may be installed (default: warn and permit for a first pull, refuse for a brain previously seen signed), how many valid signatures a head must carry (default: one), whether a propose-scoped head may be treated as the current state (default: no), and which trust-root digests are pinned. What is not configurable is whether the result is reported: an implementation MUST NOT present an unverified brain as verified, whatever the policy allowed it to install.

8.10 Three cases, worked through

The rules above are short, and their consequences are not obvious until they are run against a situation. Three cases cover what a deployment meets: admitting an owner, a key admitting itself, and starting from nothing.

In the first two, the brain is at revision 1 with keys *A* and *B*, both holding govern, a `govern_quorum` of 2, and a chain reaching S7. A consumer has pinned the digest of revision 1.

Case 1: admitting a third owner. *C* generates a key pair and sends the *public* half. The channel need not be confidential, though the recipients need independent grounds to believe the key is *C*'s, which the protocol does not answer. What it does offer is a place to record the conclusion once those grounds exist: the entry admitting *C* MAY carry a subject (Section 8.4), so the governance act that admits the key and the statement of whose key it is are the same quorum-approved change. A trust root at revision 2 lists *A*, *B* and *C*; a snapshot S8 names S7 as its parent, carries that trust root, and leaves every module root equal to S7's, because a revision changes nothing else (Section 8.4). *A* and *B* each sign S8.

When *C* later commits S9 and signs it, a consumer resolves five questions, and the last is where it terminates:

1. Does the structure recompute? Integrity, requiring no trust (Section 8.1).
2. Does *C*'s signature verify over S9's canonical bytes?
3. Which trust root is in force at S9? The one its first parent names, which is revision 2, and *C* is listed there.

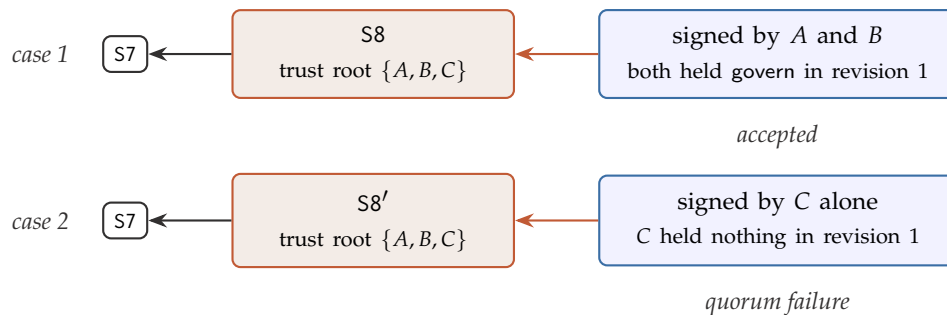


Figure 7: The same key list, admitted two ways. Both snapshots recompute correctly and both carry a signature that verifies, so integrity separates them not at all. What separates them is whether the snapshot introducing the new list carries approvals from keys that were already authorized, which an attacker cannot produce without the corresponding private keys.

4. Which scope did S9 require? Its difference from S8 shows a non-canonical composition that gained blocks, so *commit*, which C holds.
5. Why is revision 2 itself trustworthy? Because S8 introduced it carrying two signatures from A and B, who held govern in revision 1, whose digest matches the pin.

C commits and signs normally. What makes those signatures carry weight is that an earlier snapshot in the same chain authorized the key, signed by parties already authorized, and that authorization travels inside the brain, so it reaches every consumer without an extra fetch.

Case 2: a key admitting itself. C installs the brain, which is public and requires no permission, edits the trust root to include their own key, and builds S8' naming S7 as its parent. C signs it with their own key.

The first two checks pass: the structure recomputes, because C recomputed it correctly, and the signature verifies, because C produced it. *Integrity does not detect this attack*, which is why this section exists.

The third and fourth checks fail:

- The trust root in force at S8' is the one its parent S7 names, which is revision 1, holding only A and B. C's key is absent: *unauthorized key*.
- S8' changes the trust root relative to its first parent, so it is a revision, and requires two signatures from keys holding govern in revision 1. It carries none: *quorum failure*.

The comparison with Case 1 is sharper than it first appears (Figure 7):

The key list in Case 1 and the key list in Case 2 may be **byte-identical**. The same three keys, the same scopes, the same serialization. What distinguishes them is not the list. It is *who signed the snapshot that introduced it*.

An attacker can write any key list they like; what they cannot write are its approvals, because producing one requires a private key they do not hold. Two consequences follow. Rejection propagates forward: a later snapshot naming S8' as its parent draws its trust root from a snapshot already rejected, so the whole subtree falls with it. And this case fails *without any pin at all*, provided the consumer holds the chain, because the verifier is checking an internal transition rule rather than an external anchor: the chain the attacker must ship contains the evidence that their own change of authority was never approved.

Case 3: starting from nothing. There is no prior state, so nothing can be derived and the first authority is declared. *A* and *B* generate their key pairs; a trust root at revision 1 lists both with a `govern_quorum` of 2; a genesis snapshot is built with an empty `parents` list, that trust root, and empty module compositions; and both keys sign it, satisfying the quorum the trust root itself declares (Section 8.6).

Read from inside the artifact, this proves nothing. In the first two cases the outcome was decided by a rule a verifier could check against material it already held; here there is no such material, and any party can construct an indistinguishable object.

What gives a genesis meaning is therefore the step outside it: the digest of revision 1 is published through an independent channel and pinned by consumers on first contact (Section 8.7). From that moment the derivable rules take over. *A genesis is not validated, it is anchored*, and it is the only object in this protocol of which that is true.

8.11 Which impersonation routes remain closed

Table 12 judges the design against the attacks it is meant to stop, including the two it does not. One mechanism closes most of them: a trust-root revision is not a claim about the key list but a document that must carry the signatures of keys authorized in the previous list.

Table 12: Impersonation routes against a signed brain. The last two rows are honest limitations rather than defences.

Route	Outcome
Add blocks, recompute roots, sign with the attacker's own key.	Rejected: the key is absent from the trust root in force.
Add blocks and strip the signatures.	Rejected for any consumer that has seen this brain signed; reported as unsigned rather than as valid.
Add blocks and ship an attacker-authored trust root listing their key.	Rejected: the trust root's digest does not match the pinned one.
Add the attacker's key to the existing trust root.	Rejected: the revision requires <code>govern</code> signatures from keys authorized in the previous revision, and the attacker holds none of their private keys.
Reuse a legitimate signature over modified content.	Rejected: the signature covers the snapshot's canonical bytes, and modified content is a different snapshot.
Recombine legitimately signed module roots into a state that never existed.	Rejected: signatures cover snapshots, not roots, and no signed snapshot names that combination.
Backdate a signature made with a stolen key.	Rejected: validity is positional, so a <code>compromised_from</code> record withdraws trust by chain position and a written timestamp decides nothing.
Serve an older, legitimately signed snapshot indefinitely.	<i>Not prevented.</i> Withholding is a freshness problem, not an authenticity one (Section 14).
Present a fabricated brain to a consumer who has never seen the genuine one.	<i>Not prevented.</i> First contact is irreducible (Section 8.7).

8.12 Two deployments, one format

Nothing above changes with the size of a deployment; only its configuration does. Two common cases exercise opposite ends of the same structure.

A brain with a single author. One key, holding every scope, with `govern_quorum` of 1. The trust root has one entry, signing is one command at publication, and what it buys is not access control but tamper-evidence: a brain republished by anyone else fails verification for every consumer who has pinned this one. Adding a second key later is an ordinary trust-root revision signed by the first.

Public read, restricted write. Anyone may install the brain; a few may publish it. Reading is a registry concern and requires nothing from this section, and the trust root governs whose authorship a consumer *accepts* rather than who is physically able to write. Maintainers hold `commit`, a subset holds `govern`, canonical drops are restricted to keys holding `drop:canonical`, and recurring outside contributors MAY be listed with `propose`, which is enough for their snapshots to be marked as proposals and not enough for one to be mistaken for the published state. A stranger needs no listing at all: SSHSIG embeds the public key, so a snapshot signed by a key the trust root has never seen is *attributable*, its author identified by the signature with no authority attached, and Section 13 defines how such a proposal is judged. What requires a governance act is granting authority, never being heard.

8.13 What this deliberately does not solve

With the above in place, a hostile registry cannot fabricate state: everything verifies offline against the pinned trust root. It can still *withhold*, serving an older, legitimately signed snapshot indefinitely. Signing does not address this, and Section 14.3 states it as a limitation.

Confidentiality is out of scope. Nothing in this version encrypts knowledge, controls who may read a module, or supports selective disclosure. A brain is distributed whole, per module, to anyone the registry lets read, and the trust root governs whose writes a consumer accepts, never who reads (Section 8.4). The consequence for retention is concrete: crypto-shredding is not a removal mechanism here, because there is no encrypted-block scheme for it to act on, and a deployment whose content must be unreadable to some parties has a requirement this version does not meet. Section 16 carries it as an open question.

9 The Boltzmann Protocol

An installed OCI Artifact is passive data. The *Boltzmann Protocol* is the contract through which any conforming client interacts with a brain, along two paths, query and ingestion. Boltzmann Brain is therefore delivered as a protocol rather than as a deployable service or container image: the brain is portable data, and any client that speaks the protocol can read and extend it. The protocol includes no LLM of its own; it governs structure, identity, validation, indices, snapshots, and distribution.

The operations divide into five *roles*, and a role is both the unit of conformance (Section 15) and the unit of capability: a client implements what it claims and no more. Table 13 states the surface.

Because these are protocol operations rather than a bundled engine, they can be implemented independently and in any language, and the brain never depends on a particular process being kept running. `pack` in particular is defined to produce the same on-disk layout that a registry would serve, so that any OCI tool can copy the result without implementing this protocol at all.

Table 13: Protocol operations by role. Every operation is defined over an installed snapshot; none of them requires a running service.

Role	Operation	What it does
Reader	snapshot	Report the current snapshot and its installed modules.
	root_of	Report the Merkle root of one installed module.
	open_index	Open an index of a named kind over a module.
	resolve	Resolve a block_id to its bytes, verifying the hash.
	prove	Produce an inclusion proof of a block in a module: sibling hashes, leaf index, and tree size.
	resolvability	Report which blocks are resolvable and which are tombstoned.
	verify	Recompute the whole hash structure of an installed snapshot (Section 8.9).
	browse	Walk an axis of the catalog and list what is filed under a class (Section 6.7).
Authenticity	search	Answer a query with an Evidence Bundle (Section 11).
	sign	Produce a detached signature over a snapshot under the protocol namespace.
	authenticate	Check signatures against the trust root in force at a snapshot's position.
	pin	Record a trust root digest as the anchor for this brain.
	rotate	Commit a trust-root revision, under the quorum rule.
Writer	revoke	Record that a key is retired, or compromised from a chain position.
	init	Create a brain: a genesis snapshot with no parents and its first trust root (Section 8.6).
	register	Preserve a source as canonical evidence, with provenance.
	replace	Register a new edition and record supersession.
	define_task	Emit a processing task for an external model.
	validate	Judge candidate blocks against the protocol's checks.
	classify	File canonical blocks under classes on a declared axis (Section 6.7).
	commit	Incorporate validated candidates and advance the snapshot.
	merge	Reconcile two histories into a snapshot naming both as parents.
	rebase	Replay a history onto another, minting new snapshot identities.
Retention	squash	Collapse a run of snapshots into one, preserving provenance.
	drop	Exclude blocks from a module, cascading through provenance.
	drop_by_producer	Exclude everything a given model, pipeline, batch, or actor produced.
	supersede	Record that a newer block takes precedence.
	demote	Reduce a block's retrieval priority without removing it.
	prune	Reclaim blocks no retained root still references.
Distribution	redact	Destroy bytes a retained root still names, under policy.
	pack	Materialize the current snapshot as an OCI Artifact locally.
	push	Publish to the current branch's tag, refusing to overwrite a diverged remote and reporting a publish another one replaced (Section 7.4).
	pull	Install an artifact, optionally only some modules, into the branch its tag names.
	fetch	Retrieve a remote history without moving the local pointer (Section 13.6).
	branch	Create, list, or delete a named branch, locally or as published (Section 7.5).
	checkout	Make another branch current, recording the current head under its own.
	join	Bring another branch into the current one: fast-forward, or reconcile.

Protocol version. Every block envelope, composition document, snapshot, and published manifest declares the protocol version it conforms to. A client that meets a document declaring a version it does not implement MUST refuse it and say so. This is a coarser check than schema versioning (Section 6.9) and answers a different question: not “can I read this knowledge” but “is this a Boltzmann artifact at all”.

Failures are typed. An implementation MUST distinguish, and report distinguishably, at least the conditions in Table 14. Collapsing them into a generic error defeats guarantees stated elsewhere in this document: a consumer that cannot tell a redacted block from a corrupted one has lost the property Section 12.5 requires, and one that cannot tell an uninstalled module from an empty one will silently answer queries from a subset of the brain it thinks it has.

Design rule

The LLM never writes directly to the Merkle DAGs or to the indices.

10 Ingestion via an External LLM

Ingestion separates the preservation of a source from its interpretation: the original is registered deterministically, then semantic processing is delegated to the user’s LLM.

10.1 Canonical registration

Canonical evidence enters through three protocol operations. There is no in-place edit of a source: a new edition is always a new block.

Register.

1. The user provides a PDF, text, repository, conversation, image, or other source.
2. The LLM invokes the `brain.ingest` operation through the available skill or adapter.
3. The format is validated, its SHA-256 computed, and exact duplicates detected; re-registering an identical blob is a no-op.
4. The original is stored as an immutable canonical block, optionally together with a normalized view produced by a named deterministic pipeline.
5. The protocol records who incorporated it, when, from where, and under what license or policy. The actor is the party on whose account the operation runs, which is ordinarily the person the adapter is acting for and not the model that called it; the model and the harness it ran inside are recorded as assisting parties (Section 5). An implementation that recorded the model as the actor, or omitted it entirely, would answer “who did this” with the wrong half of the answer.
6. The canonical Merkle root changes and a new snapshot becomes available, even though no semantic interpretation exists yet.

Registering a source does not declare it true. The canonical module asserts that the evidence was incorporated and preserved.

Table 14: Failure conditions a conforming implementation must distinguish, grouped by what a caller can do about them.

Condition	Meaning
<i>Identity and integrity</i>	
Unknown schema	The block's schema version is not implemented here.
Block not found	The identity is not held by this brain.
Block tombstoned	The identity is known; the bytes were destroyed by redaction.
Integrity failure	Stored bytes do not hash to the identity they claim.
Inclusion proof failure	A membership proof does not reconstruct the root.
Membership failure	A returned block is not in the installed snapshot.
<i>Scope of the installed brain</i>	
Module not installed	The snapshot does not name this module.
Stale index	A travelling index is bound to a root other than the installed one.
Append-only violation	A drop was attempted on the episodic module.
Policy refusal	The retention policy does not authorize this removal.
<i>Authenticity (Section 8)</i>	
Unsigned brain	A signature was required and none is present.
Signature invalid	The signature does not verify over the snapshot's canonical bytes, or was made under another namespace.
Unauthorized key	The signing key is absent from the trust root in force, on a snapshot presented as the current state; the same key on an offered proposal is reported as attributable instead (Section 13).
Insufficient scope	The key is authorized, but not for the change this snapshot made.
Retired key	The key was authorized once and not at this position; earlier signatures by it stand.
Compromised key	The key's signatures are withdrawn from a recorded position onward.
Quorum failure	A trust-root revision lacks the required govern signatures from the previous revision.
Trust root mismatch	The pinned revision is not reachable from the trust root in force through quorum-satisfying revisions.
Revision regressed	The revision in force is an ancestor of one this consumer already validated; governance walked backward.
Chain truncated	The first-parent ancestry cannot be resolved back to the anchor; authority above the deepest resolvable snapshot is undecidable, integrity is unaffected.
<i>Reconciliation (Section 13)</i>	
Divergence	The remote is not an ancestor of the local snapshot.
Rollback	The served head is a strict ancestor of the head this consumer already holds; the remote walked backward.
No common ancestor	Two histories share no ancestor, so no three-way merge is defined.
Multiple merge bases	Criss-cross histories yield more than one best common ancestor, so the three-way merge is not determined.
Governance conflict	The histories being reconciled carry different trust roots.
Lost publish	A tag re-read after publishing names a snapshot that is neither the one published nor its ancestor; another publish replaced it.
Reconciliation open	A reconciliation is in progress, and the operation would move or record the head it is stated against.
<i>Branches (Section 7.5)</i>	
Branch not found	No ref of that name is held, locally or as a published tag.
Branch exists	A ref of that name is already held.
Invalid branch name	The name violates the grammar, is reserved, or maps to a tag

Replace. When a newer edition of a source should take precedence, the protocol registers the new original as a distinct block and records a supersedes relation to the previous one. That relation is a provenance edge rather than a field of either block, so both remain pure statements about the bytes they preserve and the precedence between them stays auditable where every other actor-dependent record lives. The caller chooses whether the superseded original remains in the current composition, kept for audit, or is dropped in the same commit; either way the new block is what subsequent processing tasks read. `replace` is `register` plus a supersession edge, never a mutation of bytes already stored.

Drop. Excluding a canonical block from the current composition is the same drop operation defined in Section 12, with a privileged cascade: every derived block that cites the dropped evidence is dropped by default in the same commit, unless the caller has already registered a replacement and requests re-derivation against it. Dropping canonical evidence is allowed only under explicit policy (Principle 2).

10.2 Processing task

After canonical registration, the protocol returns a structured task to the LLM. The task identifies the source, the instructions, the allowed memory types, the required references, and the exact output schema:

```
{
  "operation":      "extract_knowledge",
  "source":         "sha256:PDF123",
  "allowed_memory_types": ["episodic", "semantic", "procedural"],
  "requirements":   ["cite source ranges", "do not invent"],
  "output_schema":  "boltzmann.candidates/v1",
  "task_id":        "ingest-2026-07-24-01",
  "instructions":   "..."}
}
```

The LLM reads the source and returns typed candidate blocks. A document may produce semantic and procedural memory without producing episodic memory; the classification depends on what the source actually contains.

Two operations are defined. `extract_knowledge` asks the model to read a registered source and propose typed blocks from it. `rederive` asks it to rebuild derived blocks against a replacement canonical source, which is the operation a caller requests when dropping evidence that has since been superseded (Section 12.3).

The resulting provenance records cite the `task_id`, so a later batch invalidation can name this task as the producer of everything it yielded.

Only three memory types are proposable

A processing task **MUST NOT** allow candidates in the **canonical** or **provenance** modules, and an implementation **MUST** reject a task that does. Canonical registration is deterministic: it preserves observed bytes and needs no interpretation. Provenance is written by the protocol itself. Opening either to a proposer would put the external model in charge of the evidence or of the audit record, which is precisely the boundary of Section 9.

Wire schemas. Three schema identifiers are normative, and each is carried explicitly in the document it governs so that a receiver never has to infer it: `boltzmann.task/v1` for the processing task, `boltzmann.candidates/v1` for the model's response, and `boltzmann.evidence/v1` for the Evidence Bundle (Section 11).

10.3 Validation and commit

Results from the LLM do not enter automatically. Validation is the point at which a *proposal* becomes a *block*: before it, a candidate is untrusted data with no identity; after it, an accepted candidate has a `block_id` and is eligible for commit. No other operation may mint a block.

Every candidate receives exactly one of four verdicts (Table 15), and only `VALIDATED` candidates MAY be committed. A verdict MUST be accompanied by the issues that produced it, each naming the check that fired and, where applicable, the field within the payload, so that a rejection is actionable. A verdict is not a write-path ephemeral, either: every block that enters a composition MUST be accompanied by a *validation* record in provenance (Section 5) naming the verdict, the checks that ran, and the task, so that “it was validated” is a claim a consumer verifies in the signed composition rather than takes from whoever committed. Recording the verdicts of candidates that were never committed is OPTIONAL.

Table 15: The four validation verdicts.

Verdict	Meaning
<code>VALIDATED</code>	Well-formed, referenced, and consistent. Eligible for commit.
<code>PENDING_REVIEW</code>	Admissible, but not decidable by the protocol alone. Requires a human or a policy decision.
<code>REJECTED</code>	Malformed, unreferenced, or duplicate. Never committed.
<code>CONTRADICTED</code>	Well-formed but in conflict with knowledge already held; the conflicting blocks are named.

The checks. A conforming implementation MUST apply at least the following. *Schema*: the payload satisfies a registered schema for its declared memory type. *Allowed type*: the candidate’s memory type was permitted by the task. *Evidence*: every cited canonical block exists in the installed snapshot. *Evidence consistency*: the citations a candidate states match the source the task named, since a payload claiming different evidence is a validation failure and not something to reconcile silently. *Content*: bytes a candidate names are present and hash as claimed. *Duplicate*: the resulting identity is not already in the composition. *Relation*: referenced blocks exist and the relation is admissible. *Contradiction*: the candidate does not conflict with knowledge already held.

How contradiction is detected is left open, and a check that cannot decide MUST yield `PENDING_REVIEW` rather than guess in either direction. The protocol fixes what the verdicts mean and which of them may be committed, not how an implementation reaches them.

For each accepted block, the protocol prescribes the following steps:

1. Serializes the block canonically.
2. Computes its `block_id` by hashing.
3. Stores the block as an immutable object.
4. Connects it to the source via provenance.
5. Incorporates it into the episodic, semantic, or procedural Merkle DAG.
6. Updates B^+ trees, lexical, vector, and graph indices as appropriate.
7. Creates a new snapshot and, when published, a new OCI version.

11 Query via an External LLM

For now, Boltzmann Brain is conceived as a medium for LLMs. The user converses with their usual LLM; a skill or adapter lets that LLM query the brain through the protocol. The brain returns raw or semi-raw knowledge, and the LLM turns it into an answer appropriate for the task.

11.1 Query flow

1. The user asks the external LLM a question.
2. The LLM interprets the intent and calls `brain.search` with the query and optional filters.
3. The protocol combines B^+ trees, lexical search, vectors, and graph.
4. Knowledge blocks and their canonical sources are retrieved.
5. The protocol verifies hashes and membership in the installed snapshot.
6. An Evidence Bundle is returned.
7. The LLM explains, summarizes, compares, or uses the evidence.

11.2 Query planning and guarantees

Step three above hides a query planner. By Principle 7 a query is declarative: the caller expresses intent (a query, optional filters over memory type, class, or recency, and optional hints such as a retrieval mode or a result limit) and never names a physical index. Choosing which indices to use, and how to combine them, is the responsibility of a conforming implementation, in the same way that the SQL standard defines a query language and its semantics while each database ships its own optimizer. Selection follows the shape of the query, over the indices of Table 6. In practice retrieval is hybrid: filters narrow a candidate set, lexical and vector search generate candidates in parallel, their rankings are fused, the graph expands from the strongest hits, and the hash map resolves and verifies each result.

The protocol fixes the contract and the invariants, not the algorithm. A conforming implementation must return knowledge blocks with their provenance and a retrieval score, never prose; every returned block must be verified against the installed snapshot by hash and membership; and no single index may be treated as authoritative. The fusion method, ranking weights, graph expansion depth, and any adaptive intent classification are deliberately left to the implementation. The guarantee has a name and a boundary. What this contract delivers is *soundness*: every returned block is genuinely in the installed snapshot, and a consumer can re-check the claim, because hash and membership verification recompute from the bytes. What it does not deliver is *completeness*: no retrieval mode, exact resolution included, promises that everything in the snapshot satisfying the query was returned, and no mechanism in this protocol lets a consumer detect that a relevant block was withheld; the truncated field of the Evidence Bundle is a responder's assertion, not a check. Ranking sits inside the same boundary: the protocol guarantees *verifiability*, *not identical ranking*, and two conforming implementations may return the same verifiable set of blocks in a different order, because vector search is approximate and ranking is tunable. Filters sharpen the distinction rather than escape it: a filter **MUST** be honored on what is returned, and honoring it says nothing about what was withheld.

The request. A query separates three things: the terms, the conditions that narrow the snapshot, and the advice a planner may follow or ignore.

```
{
  "text": "how do I compute Fourier coefficients",
  "filters": {
    "memory_types": ["semantic", "procedural"],
    "classes": ["sha256:CLASS_FOURIER"],
    "since": "2026-05-01T00:00:00Z",
    "until": null,
    "tags": ["exam"],
    "evidence": ["sha256:PDF123"],
    "include_superseded": false
  },
  "hints": { "mode": "auto", "limit": 10, "expand_depth": 1 }
}
```

The text *MAY* be empty. “The episodes of last May” is a complete request with no terms in it, and refusing it would make recency and class filters unusable on their own.

Filters are conditions and *MUST* be honored. Hints are advice: the retrieval mode, the result limit, and the expansion depth express what the caller believes will work, and an implementation that ignores every hint and still returns verified blocks with their provenance is conforming. The named modes are auto (infer intent from the query’s shape), exact (resolve identities only), lexical (match terms as written), semantic (match meaning, accepting approximation), and associative (expand outward from the strongest hits along declared relations). A mode names a strategy, never an engine, which keeps Principle 7 intact.

The classes filter names class blocks (Section 6.7), which makes “the past exams on Fourier” a condition the protocol can honour instead of free text a responder may read as it likes. The `memory_types` filter is the mechanism behind R2. The `include_superseded` filter reflects that superseded blocks stay in the composition and remain verifiable; what supersession changes is accessibility (Section 12).

11.3 Evidence Bundle

The brain’s result is not a written answer. It is a data contract carrying the matching blocks and everything needed to audit them:

```
{
  "matches": [{
    "block_id": "sha256:CONCEPT1",
    "memory_type": "semantic",
    "content": { ... the block's payload ... },
    "score": "0.87",
    "sources": [{ "block_id": "sha256:PDF123",
                  "locator": "p.147",
                  "citation": "verified" }],
    "verified": true,
    "resolvable": true,
    "superseded_by": null
  }],
  "verified_against": { "semantic": "sha256:SEMANTIC_ROOT_V7" },
  "authorship": {
    "state": "authorized",

```

```

    "snapshot": "sha256:SNAPSHOT",
    "key": "SHA256:5tS7wqf2K...",
    "subject": "alex@example.org",
    "trust_root": "sha256:TRUST_ROOT_R3",
    "pinned": true
  },
  "truncated": false
}

```

Six fields carry guarantees stated elsewhere in this document, and a conforming implementation **MUST** populate all of them.

`verified` asserts that both the block's hash and its membership in the installed snapshot were checked. `verified_against` names the module roots that membership was checked against, so the assertion is re-checkable rather than taken on the responder's word; without it, `verified` would be a claim a consumer cannot audit.

`resolvable` distinguishes a block whose bytes are still present from one that was redacted. A consumer **MUST** be able to tell a removed block from a corrupted one (Section 12.5), and this is where that requirement is met at query time.

`superseded_by` names a newer block that takes precedence, so a caller that asked to include superseded blocks can tell which ones they are.

`citation` states what became of each cited source, because verification does not otherwise survive selective installation: a bundle's sources point at canonical blocks, and the canonical module may not be installed at all (Section 7.2), so a consumer can receive a fully verified semantic block citing evidence it cannot check. Since R1 makes canonical evidence the root of traceability, the bundle **MUST** say which of three situations holds: `verified`, the cited block is present in the installed canonical composition; `uninstalled`, the canonical module is not installed here and the citation could not be checked; or `missing`, the canonical module *is* installed and the citation is not in it, which is a violation of R1 and **MUST** be reported rather than smoothed over. Collapsing the last two would hide a broken brain behind an incomplete install. What `verified` does not assert is as load-bearing as what it does: it says the evidence *exists*, never that the derived claim *holds over it*. The locator is free text no check inspects, and whether the cited region supports the block's content is a semantic question outside this protocol, the same boundary Section 14.1 draws for content. Traceability, reaching the bytes a claim names, is what R1 delivers; auditing the claim against them is work the protocol hands to whoever holds the bundle.

`authorship` reports the second of the two checks of Section 8.9, separately from `verified`, which reports the first. Its state is `authorized` when a signature over the installed snapshot `verified` under a key holding the scopes that snapshot's change required, `unsigned` when the brain carries no signature at all, and `unauthorized` when a signature was present and failed any of those conditions. A conforming implementation **MUST NOT** fold `authorship` into `verified`: a bundle that collapses the two cannot express "intact, and signed by an authorized key" as distinct from "intact, provenance unknown", and a caller has no way to recover the difference afterwards.

`score` is a decimal *string*, for the same reason payloads forbid floats (Section 6.2): a number whose textual form varies across languages does not belong in a wire format meant to be compared. The scale is not normative; only the encoding is.

The consumer receives block identities, content, memory type, sources, and verification status, and remains free to explain, summarize, compare, or reuse the evidence. Those transformations happen outside the brain.

12 Retention, Forgetting, and Erasure

Everything described so far only adds. Content addressing and immutable blocks make individual units append-only, but a module version is a *composition* of those units, and compositions can exclude what should no longer belong. Without an explicit way to drop blocks, a brain that only grows is neither practical nor always permissible for its operator, and it cannot recover from knowledge that was deliberately wrong.

12.1 Why exclusion must be first-class

Three pressures call for different responses, and they should not be conflated.

Wrong or obsolete knowledge. A semantic definition may be incorrect, a procedure may be outdated, or a canonical source may have been ingested in error. Leaving the block in the current composition and merely down-ranking it is not enough: the wrong claim still lives in the snapshot that consumers install. The protocol needs a way to *exclude* it from the next version.

Cost and signal. Unbounded growth costs storage and precision. Stale near-duplicates compete for the top of every ranking, so a brain that never excludes gets measurably worse at answering.

Obligations to destroy. Credentials, private records, or licensed material may have to disappear even from retained history: a deployment can be under obligations, contractual, organizational, or imposed by whoever governs it, that demand destruction rather than exclusion. That case is stricter: it requires destroying bytes that a retained root still names, and it is treated separately in Section 12.5. One boundary holds for this whole section: the protocol states precisely what each mechanism does and does not destroy, and whether that satisfies any particular obligation a deployment is under is a determination this specification does not make.

12.2 What biology suggests

Section 3 took forgetting to be a regulated process rather than a storage failure, and cognitive psychology separates the availability of a trace from its accessibility [4]. The architectural reading is that exclusion from the current composition is the default, that reclamation of what no retained version needs is a background process, and that destroying evidence is a rare, policy-bound exception rather than the ordinary cleanup path.

12.3 Drop: excluding blocks from a module

The primary removal operation is *drop*. It applies to the **canonical**, **semantic**, **procedural**, and **provenance** modules. The **episodic** module is exempt: episodes are a chronological record of what happened, so corrections append new episodes or supersession relations (Section 5).

A drop does not mutate blocks in place. Blocks remain content-addressed and immutable. What changes is the *composition* of the module: the protocol builds a new Merkle DAG whose leaves are the surviving *block_ids*, computes a new root, rebuilds the module's indices from that composition, and records the removal in provenance (Figure 8). The dropped blocks become unreachable from the new root and are eligible for pruning once no retained root references them. Structural sharing still applies: unchanged blocks are reused by hash, so the cost of a drop is proportional to what was removed, not to the size of the module (Section 6).

Example

semantic-root v1 contains K1, K2, and K3, where K2 is a wrong Fourier definition. A drop of K2 yields semantic-root v2 over K1 and K3 only. Indices are rebuilt without K2, and provenance records that K2 was dropped, by whom, and why. Optionally the external LLM re-derives a corrected K2' from the canonical source and a later commit adds it under v3. The wrong block never appears in v2 or v3.

Provenance cascade. Dropping a block is not always local, and the cascade differs by module.

Dropping a *semantic* or *procedural* block removes it from that module's composition and rewrites or removes provenance edges that pointed to it. Downstream dependents are treated the same way. The canonical source remains, so the caller may later re-derive a corrected block from the same evidence.

Dropping a *canonical* block is privileged. Because derived knowledge cites that evidence as its root, the protocol always walks the dependency closure and, by default, *drops* every semantic and procedural block that listed the canonical as evidence, in the same commit. Re-derivation is not the default: it runs only if the caller has already registered a replacement canonical (replace) or explicitly requests *rederive* against another canonical still in the composition. Each affected module receives a new Merkle root, so a single logical removal of evidence can publish several new module versions at once. Provenance records the actor, the policy, the reason, and the resulting roots, so the removal stays auditable even though the dropped content is no longer part of the current composition.

Example

The wrong lecture PDF was ingested as PDF_wrong, and several semantic blocks cite it. A drop of PDF_wrong yields a new canonical root without that blob; the cascade drops the derived definitions in the same commit. Registering the correct PDF and running a processing task then produces new candidates under a later snapshot. The wrong source never appears in the current composition, and neither do interpretations that depended only on it.

The cascade does not weigh evidence, and cannot. The cascade excludes every derived block that cited the dropped evidence, without distinguishing a block supported by one source from a block supported by three. The obvious refinement, dropping a derived block only when it loses *all* of its evidence, is not implementable here. Citations live in the payload, and therefore inside `block_id` (Section 6.1). A semantic block citing three sources that loses one cannot “keep the other two”: a block with two citations is a different block, with a different identity. A block whose citation set no longer holds cannot stay, whatever fraction of that set survived.

Rebuilding such a block against its surviving evidence therefore costs a full round trip through the external model, and whether a cheaper deterministic path should exist is stated as open in Section 16.

Batch invalidation. Because provenance records what produced each derived block (Section 5), a drop may be stated over a *set*: everything produced by a given ingestion, or everything derived by a given model. That is the natural response to deliberately wrong knowledge introduced in bulk, and it reuses the same cascade rather than inventing a second mechanism.

The granularity is the identity, not a release of it. Records at schema version 1 carry a producer with a version string and a drop may name it; from version 2 the version is gone, and a drop over a model reaches everything that model made. The narrowing is deliberate. A version is the field most likely to be wrong — invented by whoever filled the record in, or simply absent — and a batch invalidation that silently missed blocks because two records spelled a release

differently is a worse failure than one that reaches further than strictly necessary. Where a deployment genuinely needs to separate two releases, they are two identities. It reaches exactly the modules drop reaches: for poisoned episodic content, out of its reach by construction, Section 14.2 states the remedy.

12.4 Companion mechanisms

Drop is the ordinary way to remove knowledge from a non-episodic module. Three companion operations cover accessibility, reclamation, and the rare case where bytes must disappear while a retained root still names them (Table 16).

Supersession and demotion. A block is marked as superseded by a newer one, or its retrieval priority decays. It stays in the current composition and remains verifiable; what changes is accessibility. This is the default path for the episodic module, and an optional soft path elsewhere when history inside the current snapshot must remain queryable under an explicit filter.

Pruning of unreachable blocks. A brain retains a set of roots (tagged releases plus recent snapshots). After drops, former leaves are unreachable from those roots and can be reclaimed by mark-and-sweep [6]. Pruning never decides *what* to forget; it reclaims what no retained composition still needs.

Redaction of referenced content. Only when content that a retained root still references must be destroyed does the protocol remove bytes: a *tombstone* deletes the bytes and leaves an auditable record, or a *lineage rewrite* rebuilds history so the block was never referenced. Crypto-shredding, destroying a key instead of the ciphertext it protects [23], is deliberately not among them: it presupposes an encrypted-block scheme, and this version defines none (Section 8.13). Redaction is the path for destroying retained canonical bytes under policy. It is not the cleanup path for a wrongly ingested source or for wrong semantic knowledge: those are dropped.

Append-only is not an exemption from destruction. The episodic module is exempt from drop, and the obligations named earlier in this section can require an episode's bytes to disappear. Those two facts meet in an uncomfortable place, and this document says so plainly: for bytes inside an episode that must be destroyed, redaction is the *only* available path, so the mechanism framed here as rare and exceptional is the ordinary one for the module most likely to attract such obligations. Three consequences are normative. A retention policy **MUST** be able to declare episodic content redactable (Table 17). An implementation **MUST NOT** read the append-only rule as grounds for refusing to destroy bytes, since the two constrain different things: append-only governs how a record is *corrected*, not whether its bytes may be destroyed. And a deployment recording episodes about identifiable people **SHOULD** decide in advance which episodic content is redactable, rather than meet the question for the first time when an obligation arrives.

12.5 Why drop preserves a clean snapshot

A drop publishes a new composition. Consumers of the new root never see the dropped block: membership proofs, indices, and Evidence Bundles are computed over the survivors only. Older roots, if still retained, continue to verify exactly as before, because nothing about them changed. The brain therefore offers a *clean current snapshot* after every drop, which is the property that makes exclusion usable for wrong knowledge.

Table 16: Removal mechanisms and their guarantees.

Mechanism	What it removes	Reversible	Effect on current roots
Drop (non-episodic)	Membership in the module composition.	Via a later commit that re-adds.	New Merkle root without the block; indices rebuilt; provenance updated. Canonical drops always cascade to dependents.
Supersession / demotion	Accessibility, not membership.	Yes	None; block stays in the composition.
Pruning	Bytes of blocks in no retained root.	No	None; retained roots unaffected.
<i>Redaction (retained root still names the block):</i>			
Tombstone	The bytes; identity kept.	No	Root intact; block unresolvable.
Lineage rewrite	The block and its history.	No	Prior roots invalidated; new lineage published.

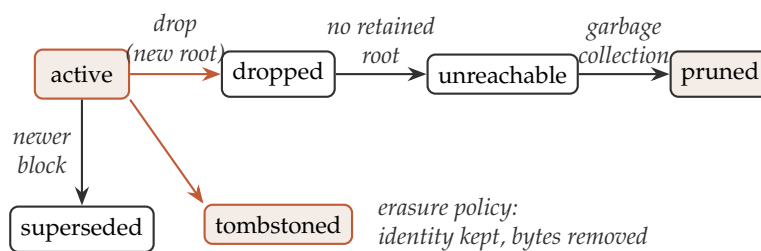


Figure 8: Lifecycle states of a knowledge block. The ordinary removal path for non-episodic modules is drop: a new Merkle root excludes the block, provenance is updated, and pruning later reclaims bytes once no retained root references it. Supersession keeps the block in the composition; redaction destroys bytes that a retained root still names.

Redaction is different: it punches a hole in a composition that still names the block. The Merkle DAG references identities, not bytes, so deleting the bytes behind a leaf changes no hash and membership still verifies, but reconstruction of that one block is forfeited. A conforming implementation must report which blocks of a snapshot are resolvable and which are tombstoned, so that a removed block is never indistinguishable from a corrupted one. Provenance is again the home for that record. The tombstone set is also part of the snapshot

itself: a module reference whose composition still names destroyed bytes MUST list those identities in its tombstones member (Section 6.5). That is what makes redaction visible to the difference a verifier computes, and therefore what makes the redact scope checkable; without it, destroying bytes changes no hash, the diff shows nothing, and the scopes table would demand a scope no difference could ever require. A redaction consequently produces a successor snapshot, like every other change of state. C2PA’s redaction of hash-referenced assertions is the deployed precedent for the shape, down to recording what was removed in the claim itself [8].

Two limitations remain, and the first is stronger than “not anonymous”. A retained `block_id` is an unsalted hash over a standardized five-key envelope, so for low-entropy payloads under a known schema the preimage space is enumerable, and an attacker holding the tombstone can recover the destroyed value by exhausting it; C2PA documents the same attack against its hashed assertions and mitigates it with a per-assertion random salt [8]. This design does not adopt the salt, because identity is its spine: a salted block is a different block, re-registering an identical original would stop being a no-op, and deduplication and structural sharing would fail exactly where they matter (Section 5). The consequence is stated instead: redaction of low-entropy content bounds disclosure rather than preventing it, and content whose later confirmation must be impossible should not be registered in cleartext in a brain built to be distributed. And erasure does not propagate by itself across already-pulled copies: the protocol can publish a revocation, but a distributed brain can only signal destruction, not guarantee it. Neither limitation applies to an ordinary drop of wrong semantic or procedural knowledge, because the new root simply never included the block.

12.6 Policy is configuration

As with query planning (Section 11), the protocol fixes the operations and the invariants, not the thresholds. What the protocol requires is that every drop, supersession, prune, and redaction be explicit, recorded in provenance, and reportable, which is the content of Principle 8. What a deployment decides is stated in a *retention policy* (Table 17), which an implementation MUST consult before performing any removal and MUST refuse to act against.

Table 17: The retention policy: what a deployment decides, and the defaults this version specifies.

Setting	Decides	Default
Droppable modules	Which modules permit drop.	all non-episodic
Canonical drop	Whether canonical evidence may be excluded at all.	forbidden
Cascade review	How many dependents a cascade may drop before the commit requires review.	no threshold
Retained roots	How many recent snapshots stay reachable, on top of tagged releases and branch heads.	10
Redactable content	Which media types may have their bytes destroyed.	none
Episodic redaction	Whether episodic content may be redacted, which is the only erasure path an append-only module has.	forbidden
Allowed mechanisms	Which removal mechanisms are permitted at all.	all the module allows

Three of these defaults are deliberately restrictive. Canonical drops are forbidden by default because excluding evidence forfeits re-derivation from that source, which Principle 2 makes

a privileged act. Both redaction settings are forbidden by default because redaction is the mechanism for destruction obligations and safety, not the cleanup path, and a deployment that has not decided which content is redactable has not decided that it is; the episodic setting is separate because it is the module where the question is most likely to arise and least likely to have been considered. The episodic module can never appear among the droppable modules: it is append-only by protocol, not by policy, and no configuration may make it otherwise.

One setting that does not exist

Whether removals are recorded is not configurable. Every drop, cascade, supersession, demotion, prune, and redaction **MUST** be written to provenance, and no policy document, however constructed, may turn that off. Auditability is the property that makes exclusion trustworthy, so exposing it as a knob would make every other guarantee in this section conditional on a setting.

The ledger is load-bearing, so it is checkable. The rule above would be honor-system if provenance could quietly lose the records themselves: the provenance module is not canonical, so dropping a removal record requires only `commit`, and the ledger Principle 8 promises would be whatever the last writer left in it. The invariant that closes this is verifier-side. For every block absent from a module’s composition relative to the snapshot’s first parent, a removal record naming that block **MUST** exist and be reachable from the current snapshot, and a verifier **MUST** reject a snapshot that fails the check. The rule needs no special case for the ledger itself: a dropped removal record is itself a block absent relative to the parent, so its own removal record is required, and silence about silence is not representable.

13 Divergence and Reconciliation

Section 7.4 requires an implementation to detect that two brains advanced from a common ancestor and to refuse to overwrite. Refusing is safe and incomplete: it leaves reconciliation to hand-editing, and it means a partial install can never be published back over the tag it came from. This section defines the reconciliation the refusal was standing in for.

13.1 One structural change

A snapshot names its predecessors in parents, a list (Section 6.5). That is the whole structural change: a field goes from scalar to list, and no new document is introduced.

Order is significant in exactly one way. The *first parent* is the history the reconciliation was performed onto, and every rule in this document that speaks of “the parent” means the first one: the scope a signature must hold (Section 8.4), the trust root in force (Section 8.5), and the difference a consumer reads as the change this snapshot made. A conforming implementation **MUST** treat the remaining parents as merged-in history and **MUST NOT** derive any authorization from them.

13.2 Why the structural part is nearly free

This is a genuine advantage over version control. Git merges lines of text; here what is merged is a set of immutable, content-addressed blocks. Nothing is ever “the same block, slightly modified”: modifying a block yields a different identity (Section 6.1). A textual conflict is therefore not representable, and reconciliation of a single module is set arithmetic over identifiers: for a fixed ancestor the merge is symmetric in the two sides, so which brain performs a reconciliation

does not change its result. What the result does depend on is the ancestor, and its selection is specified below.

What makes it three-way rather than two-way is the common ancestor, and this is the part that cannot be skipped. Without it, a block present in one composition and absent from the other is ambiguous between “they added it” and “I dropped it”, and those demand opposite outcomes. A conforming implementation **MUST** identify the best common ancestors of the snapshots being reconciled: the common ancestors that are not themselves ancestors of another common ancestor. When the histories share none, it **MUST** refuse with a distinguishable failure (Table 14). When they share more than one, which criss-cross histories produce, it **MUST** refuse with a distinguishable failure of its own: the merge yields different results for different ancestors, so an implementation that silently picked one would be choosing the outcome, and Git documents the same selection as unspecified for exactly this reason [15]. The operator resolves the ambiguity explicitly, for example by first reconciling the ancestors themselves, so that a later reconciliation has a unique base. A reconciliation proceeds only when the best common ancestor is unique, and that ancestor is the B of Equation 2.

For each module, with B the ancestor’s composition and X, Y the two compositions being reconciled, the result is

$$M = (B \cup X \cup Y) \setminus ((B \setminus X) \cup (B \setminus Y)), \quad (2)$$

which is to say: everything either side added is kept, and everything either side removed stays removed.

Removal wins, and that is a feature. Equation 2 gives precedence to exclusion, so a block one side dropped does not return because the other side still held it. A related property follows from content addressing and is worth recording: dropped evidence cannot be smuggled back in by re-ingesting it. Re-registering the same source yields the same identifier (Section 5), so the reconciliation recognizes it as something this brain removed rather than as a new contribution, with no special rule required. Deliberately re-admitting a block that was dropped remains possible and remains an explicit commit, which is where a decision of that kind belongs.

The result is a candidate, not a commit. Equation 2 is applied per module and is individually correct in each, which is why it is not sufficient: the invariants this architecture rests on run *between* modules, and a set operation never crosses that boundary.

13.3 Merge, rebase, and squash are recording strategies

This inverts the intuition anyone brings from version control. In Git the three strategies differ both in how the result is computed and in how history is recorded: a rebase replays patches, and can therefore land on a tree a merge would not have produced. Here a snapshot is a complete statement of composition rather than a patch, so there is nothing to replay sequentially, and all three produce the same set of blocks. What differs is only the lineage recorded, and therefore who remains on record as the author (Table 18).

In Git, choosing among these is a question of tidiness. Once snapshots are signed (Section 8), it is a question of attribution: rebase and squash mint new snapshot identities, so a contributor’s signature no longer covers anything in the resulting history and their work ends up signed by whoever performed the operation. That may be exactly what is wanted for a small reviewed contribution, but an implementation **MUST** report it as a consequence of the choice rather than let it happen silently. A conforming implementation **MUST NOT** present a rebased or squashed contribution as bearing the contributor’s signature.

Table 18: The three strategies produce identical compositions and different histories. Once snapshots are signed, the difference is attribution rather than tidiness.

Strategy	Lineage recorded	Same blocks	Their snapshots kept	Their signature survives
Merge	Two or more parents.	Yes	Yes	Yes
Rebase	One parent: mine.	Yes	No	No
Squash	One parent, their snapshots collapsed into one.	Yes	No	No

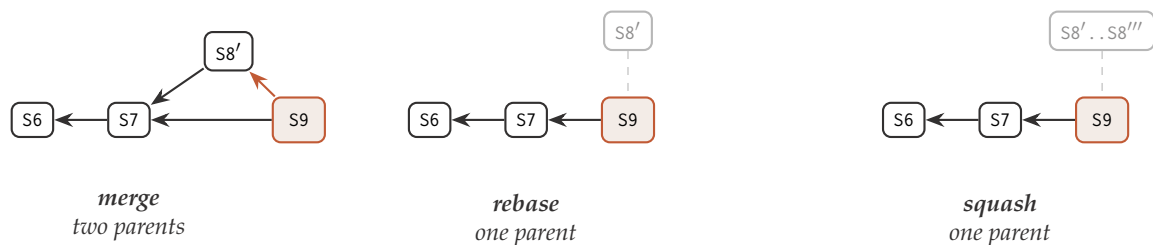


Figure 9: All three land the same blocks in S9. Only merge keeps the contributed snapshot in the history, and therefore only merge keeps the contributor's signature covering something. Greyed nodes are outside the resulting history.

Rebase and lineage rewrite are the same machinery. Replaying immutable blocks onto a new base is deterministic, but it changes snapshot identities, which invalidates signatures over them and any root already published. That is exactly the property of the lineage rewrite of Section 12, and unifying them is a simplification: both are legitimate only before publication, under version control's own rule about rewriting public history.

Squash collapses snapshots, not provenance. Squash is more useful here than in version control, because an ingestion session mints many intermediate snapshots nobody cares about individually. It MUST therefore be defined precisely or it breaks Principle 8: a squash compacts root history and MUST preserve every provenance record the collapsed snapshots produced. An implementation that discarded provenance while collapsing snapshots would be destroying the audit ledger to tidy a chain.

13.4 The conflicts that are real

Because no textual conflict is representable, the conflicts that remain are semantic, and there are few of them (Table 19). One proves that reconciliation cannot be purely structural.

Branch A drops canonical block C. Branch B adds a semantic block derived from C. Each module's set arithmetic is individually correct: the drop is respected in the canonical composition, the addition is respected in the semantic one. The result nonetheless violates R1, because a derived block now cites evidence that is not in the composition. The invariant that broke lives *between* modules, and Equation 2 never crosses that boundary. That single case is enough to establish that the reconciliation rule MUST be provenance-aware.

Table 19: Semantic conflict classes and how each is resolved.

Conflict	Resolution
One side drops canonical evidence the other side derived from.	Structurally invisible, semantically fatal. The derived block is rejected by the evidence check, exactly as the canonical cascade rejects a dependent (Section 12.3).
Both sides add blocks that contradict each other.	Not structurally detectable at all. Surfaces as CONTRADICTED when the result is re-validated, with the conflicting blocks named.
Both sides supersede the same block with different successors.	Both successors are admissible blocks and both supersession edges are recorded; the conflict is a precedence question and MUST yield PENDING_REVIEW rather than a silent winner.
The two sides carry different trust roots.	MUST NOT be reconciled automatically. Unioning two key lists grants the union of both sides' permissions, which defeats the quorum rule outright (Section 8.4). The reconciliation MUST be refused and resolved as an explicit governance act.

The rule. The structural reconciliation of Equation 2 is automatic, and its result MUST then be validated as if it were an ingestion (Section 10). Every question these conflicts raise, does this evidence exist, does this contradict what is already held, is this relation admissible, does the payload satisfy a registered schema, is already asked on the ingestion path, so no new machinery is introduced. Conflicts surface as candidates in PENDING_REVIEW or CONTRADICTED rather than as an unresolvable state. Put differently:

A conflict in this protocol is a validation failure, not a differencing failure.

Only blocks that emerge VALIDATED enter the reconciled composition, and the reconciliation MUST NOT be committed while any candidate is still PENDING_REVIEW. The verdicts are the report a maintainer acts on, and they name which parts of a contribution fit before anything is decided.

13.5 Missing evidence has two causes, and the diagnosis matters

When a reconciled block cites evidence absent from the composition, the block is rejected either way: it cannot enter, for the same reason the canonical cascade does not repair its dependents. But *why* the evidence is absent determines what the contributor should be told, and provenance already holds the answer, because it is the removal ledger (Section 5).

A removal record exists. The evidence was dropped deliberately. The contribution rests on something this brain judged wrong, and resending it would re-import exactly what was excluded. The contributor SHOULD be told not to resend.

No record, and the identity is unknown. The evidence was never here: the contribution shipped a derived block without its canonical source. The work may be perfectly good and the transfer was incomplete. The contributor SHOULD be told to resend it whole.

Same verdict, opposite advice. A conforming implementation **MUST** distinguish the two, because collapsing them discards legitimate work over a packaging mistake and offers no way to tell which happened.

Rejection is not final. If a replacement canonical source is later registered, the rederive operation of Section 10 can rebuild what was lost against it. That is a separate, deliberate step taken afterwards, and it **MUST NOT** be folded into the reconciliation: a reconciliation that quietly re-derived rejected blocks against substituted evidence would be inventing knowledge in the middle of an operation the operator believes to be mechanical.

13.6 Contribution from outside

The deployment that motivates all of this is public read with restricted write (Section 8.12). What follows is how an external contribution works end to end (Figure 10). Nothing in it requires a new kind of object, and the first five steps require nothing this document had not already specified.

1. **The contributor extends the brain.** They install it at snapshot *S7*, no permission needed because the repository is public, ingest sources locally, and commit. Their snapshot names *S7* as its parent. They sign it with their own key and publish it to a repository they control. If the maintainer has listed that key with propose, the snapshot arrives marked as a proposal; a key the trust root has never seen serves as well, because the signature itself identifies its author, and the snapshot is attributable (below). Authority is not what a contribution needs; identity is. A contribution is therefore a signed snapshot, in another repository, whose parent belongs to the maintainer's history. The parent pointer is the entire link, exactly as a branch is in version control. There is no proposal object and no pending state.
2. **They notify the maintainer.** Out of band: an issue, a message, a URL. This protocol defines no notification mechanism, in the same way version control defines no pull requests. The snapshot is the protocol-level artifact; the workflow around it belongs to the forge.
3. **The maintainer fetches it.** Because everything is content-addressed, the transfer is bounded by the delta: modules the contribution did not touch keep their digests and do not move, and within a fetched module every block reachable from *S7* is already present locally, byte for byte. A contribution of forty semantic blocks costs a fetch of the changed modules, not a brain.
4. **Nothing has changed yet.** The maintainer holds two histories locally while the published brain is untouched: fetch retrieves a remote history without moving the local pointer, which is why it is separate from pull.
5. **The contribution is judged mechanically, before any decision.** Reviewing a pull request means reading a diff; here the incoming blocks are candidates and the validation of Section 10.3 applies unchanged, so every incoming block emerges with a verdict and the maintainer learns which parts fit before deciding anything.
6. **The maintainer incorporates it** under one of the strategies of Section 13.3, re-validating the result as Section 13.4 requires, and signs the resulting snapshot with a key holding the scopes that snapshot's change requires.

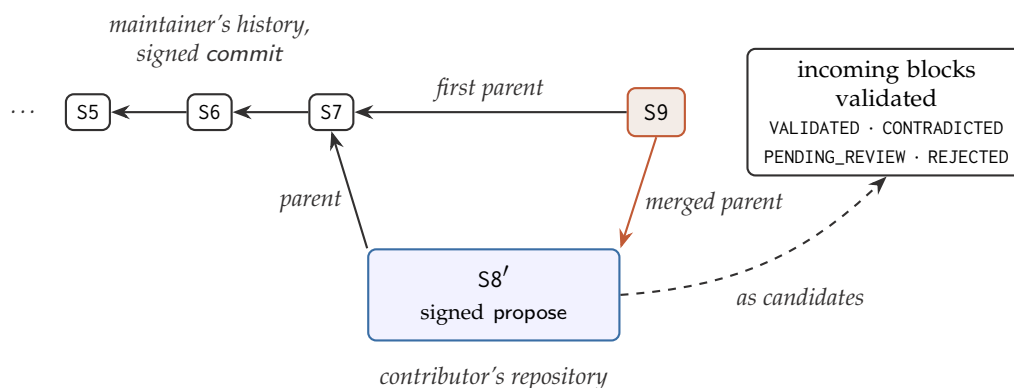


Figure 10: An external contribution needs no new object. The contributor’s snapshot names a snapshot of the maintainer’s history as its parent, which is the whole link; the maintainer fetches only the delta, validates the incoming blocks as candidates, and records the outcome as a reconciliation whose first parent is their own history.

Why propose is a scope and not a convention. A contributor’s snapshot is a legitimately signed Boltzmann snapshot in every respect, which is the problem: without a scope distinguishing it, a consumer who pulled from the contributor’s repository, or from a mirror that indexed it, would have no basis for treating it as anything other than the brain’s current state. A key holding only propose produces snapshots that are attributable, verifiable, and explicitly not the published head. A conforming implementation **MUST** refuse to treat a propose-signed snapshot as a brain’s current state unless a verification policy explicitly permits it (Section 8.9). A key absent from the trust root altogether needs one more word, because the vocabulary of Section 8 would otherwise collapse a stranger’s proposal into an attack. A snapshot whose signature verifies but whose key the trust root does not list is *attributable*: the author is cryptographically identified, and no authority attaches. The distinction is positional. Offered as a proposal, an attributable snapshot is judged exactly as a propose-signed one, block by block through validation, since candidate blocks derive no authority from their proposer anyway (Section 10.3). Presented as a brain’s current state, by a registry, a mirror, or the contributor’s own tag, the same snapshot **MUST** be reported as an unauthorized key, exactly as Case 2 demands. An implementation **MUST** distinguish, in reporting, an unknown key on an offered proposal from an unknown key on a head: the first is how an open project hears from strangers, the second is an impersonation attempt.

13.7 Publishing back from a partial install

Section 7.4 states that a partial install cannot be published back over the tag it came from, because doing so would drop the modules that were never fetched. Reconciliation resolves this rather than working around it.

A snapshot produced by a partial install names roots only for the modules that install holds. Publishing it back **MUST** be expressed as a reconciliation with the remote head, in which the modules the publisher does not hold take their roots from the remote unchanged, and the modules it does hold are reconciled by Equation 2. The published snapshot then names every module the remote named, no module regresses, and the source-snapshot annotation of Section 7 continues to record the full snapshot a projection was taken from. A conforming implementation **MUST NOT** publish a partial snapshot over a tag naming modules that snapshot omits.

14 Security Considerations

A document claiming to be a standard owes a plain statement of what it guarantees, what it does not, and where the burden sits. Much of the burden here lands on the consumer. Table 20 summarizes; the subsections that follow treat the cases where the architecture itself creates the exposure.

Table 20: What the protocol guarantees against each capability, and what it does not.

An adversary who can	Is prevented from	Is not prevented from
Modify bytes in transit or at rest	Passing any verification: every identifier is recomputed from the bytes (Section 8.1).	Denying service by corrupting what a consumer needs.
Publish a fabricated brain under someone else's name	Being accepted by any consumer holding a pin (Section 8.11).	Being accepted on first contact by a consumer with no pin (Section 8.7).
Control the registry	Fabricating state: signatures and roots verify offline. Substituting a travelling index: its digest is named by the signed snapshot (Section 6.6).	Withholding newer snapshots and serving a stale signed one (Section 14.3).
Answer queries over a brain it holds	Fabricating results: every returned block must verify by hash and membership against the installed snapshot (Section 11.2).	Suppressing results: completeness is not guaranteed, and a withheld block leaves no trace in a verified Evidence Bundle (Section 11.2).
Steal one authorized private key	Revising the trust root, when the quorum exceeds one (Section 8.4).	Signing snapshots within that key's scopes until a compromise record is published (Section 8.5).
Get content into an ingested source	Having it committed without a checkable verdict: the validation record and the producer travel in the signed composition (Section 10.3).	Having it faithfully preserved, retrieved, and replayed into a model (Section 14.1).
Read a published brain	Nothing by this protocol: confidentiality is out of scope, and a published brain is readable by whoever can pull it (Section 8.13).	Recovering approximations of dropped content from a stale travelling index (Section 6.6).

14.1 Untrusted content reaching a model is structural, not incidental

This architecture routes untrusted content into a language model by design. Canonical evidence is preserved verbatim because Principle 2 requires it, retrieved because that is what the query path is for, and handed to an external model that acts on it. Indirect prompt injection [18] is therefore a structural property of the design rather than an integration detail: a poisoned source that survives ingestion is replayed into every model that later queries the brain, and it

arrives carrying the protocol's own verification stamp attesting that it is authentic evidence. Retrieval-augmented systems have been shown to be corruptible by a very small number of injected passages [63], and content addressing does nothing to prevent it: a poisoned block is faithfully preserved, correctly identified, and honestly attributed.

The consequence has to be stated precisely, because the alternative is a false sense of safety:

What verified means, and what it does not

The verified field of an Evidence Bundle asserts that a block is authentically the knowledge that was committed to this brain, at this snapshot, by an identified key. It asserts nothing whatsoever about whether the content is true, or safe to act on. A conforming implementation **MUST NOT** describe verification as sanitization, and **MUST NOT** present canonical evidence to a model as trusted instruction.

Where the burden sits follows from the boundary of Section 9. The protocol governs what is stored and returns data rather than prose; the caller decides what to do with it. A caller that hands retrieved evidence to a model **SHOULD** treat it as untrusted input. How to mitigate what untrusted input can do, separating it from instructions, constraining the model that reads it, is the craft of whoever operates the model, exercised where the model is, with tools this protocol neither sees nor prescribes. The protocol contributes what it honestly can: every retrieved block names its canonical source, so a suspect claim can be traced to the bytes that produced it, and every derived block names its producer, so a poisoned batch can be excluded as a set.

14.2 Poisoning in bulk, and its remedy

Because provenance records what produced every derived block and who assisted in making it (Section 5), `drop_by_producer` (Section 12.3) can exclude everything a compromised model, pipeline, ingestion batch, or actor made in one operation. This is remediation and not prevention, and it should be described as such: it shortens the window during which poisoned knowledge is served, and it does not stop the poisoning. Its value is that the window is bounded by a query over provenance rather than by an audit of a corpus. The operation's reach has a boundary, and the batch case is where it bites: a drop stated over a set is still a drop, and drop does not touch the episodic module, while processing tasks may legitimately mint episodic candidates (Section 10.3). Poisoned episodes are remediated by mass *supersession* over the same provenance query, correcting the record rather than rewriting it, exactly as the append-only rule prescribes. Two consequences are stated rather than implied. Superseded episodes remain in the composition and remain retrievable under `include_superseded`, so a consumer that opts into superseded content is opting into poisoned-then-corrected content as well. And where an episode's bytes are themselves the harm, rather than wrong knowledge, the path is the one Section 12 already names for episodic content: redaction under policy, not drop.

Signatures narrow the window further in one specific way. Because a snapshot's author is authenticated, a poisoned commit is attributable to a key rather than to a declared name, and `compromised_from` withdraws every snapshot that key signed from a chosen chain position (Section 8.5). Attribution is the precondition for that response, and the operational argument for Section 8 beyond impersonation.

14.3 Freshness: the limitation signing does not remove

Everything verifies offline against a pinned trust root, so a hostile or merely stale registry cannot fabricate state. It can still *withhold*: serve an older, legitimately signed snapshot indefinitely, while concealing that newer ones exist. Every signature verifies, every root recomputes, the trust root matches the pin, and the consumer is nonetheless reading a brain that was current months ago, possibly one from before a drop that removed harmful content.

That exposure splits in two, and only one half is irreducible. Against a returning consumer, one that already holds a newer head, serving an older one is a rollback, and Section 7.4 requires refusing it: the ancestry the consumer holds is the evidence, and no party outside the artifact is needed. Revocation is where this bites, because a `compromised_from` record published in a newer head disappears from view exactly when a consumer is walked backward past it; revocation is only as fresh as the head a consumer accepts. What remains irreducible is first contact, the consumer that held nothing: nothing in this version detects withholding there, and a consumer **MUST NOT** infer freshness from a successful verification. The two known answers for that half are expiring metadata, where a signed statement asserts a validity window and its absence is itself a signal, as in The Update Framework's timestamp role [52], and an append-only transparency log, where a consumer can check that the snapshot it was served is the latest one the log has witnessed [29]. Both require a party outside the artifact, which is why neither is specified here: the offline property of Section 8.1 is the one thing this design is unwilling to trade, and both answers weaken it for consumers that cannot reach the extra party. Section 16 states this as open.

14.4 Erasure does not propagate

The limitation of Section 12.5, in security terms: a redaction destroys bytes in the brain that performs it. It does not reach copies already pulled elsewhere. The protocol can publish the fact of a removal, and a conforming implementation must report which blocks are tombstoned, but a distributed brain can only signal destruction, not guarantee it. Any deployment whose obligations require guaranteed destruction must not distribute the content whose destruction it may later have to guarantee, and no protocol feature substitutes for that decision.

14.5 Hash agility

Every digest in this version is SHA-256, and every digest is written with its algorithm as a prefix, so a future version can introduce another without ambiguity. What such a transition [3] cannot be is an in-place upgrade. A `block_id` is knowledge identity, and a Merkle root is version identity, so recomputing them under a different algorithm produces a different brain: every identifier changes, structural sharing with existing copies is lost, and signatures over old snapshots continue to cover the old values only. A migration is therefore a re-identification of a whole brain, carried out deliberately and recorded in provenance, and a conforming implementation **MUST NOT** mix algorithms within one snapshot or treat identifiers under different algorithms as comparable.

14.6 What a consumer is responsible for

The properties in this document hold for a consumer that does the following, and an implementation **SHOULD** make each of them the default rather than an option:

- Recompute integrity; never accept a publisher's assertion of it (Section 8.9).
- Pin a trust root on first contact, and refuse a change that does not satisfy the quorum rule (Section 8.7).
- Report integrity and authenticity separately, and never collapse "intact" into "authentic" (Section 8.9).
- Treat retrieved evidence as untrusted input to a model (Section 14.1).
- Treat a successful verification as saying nothing about freshness (Section 14.3).

- Treat a verified Evidence Bundle as sound, never as complete: absence of evidence in a result is not evidence of absence in the brain (Section 11.2).
- Treat citation: verified as existence and addressability of the cited evidence, never as confirmation that the claim holds over it (Section 11.3).

15 Conformance

A protocol whose conformance is asserted rather than demonstrated is a convention, not a standard. This section states what an implementation must demonstrate, and how.

15.1 Roles

Conformance is claimed per role, not for the protocol as a whole (Table 13). An implementation MAY claim the **reader** role alone, which is the common case for a consumer that installs a published brain and queries it. Claiming **authenticity**, **writer**, **retention**, or **distribution** requires the reader role as well, since each of them is defined in terms of reading a snapshot before changing it or attesting to it. An implementation that claims all five is *fully conforming*.

Authenticity is claimable separately, for a reason easy to mistake for a weakness. A consumer that recomputes integrity while holding no trust anchor is not a degraded client: it is the zero-configuration case Section 8.1 guarantees, and requiring signature verification for a reader to be conforming would make offline integrity conditional on configuration the protocol promises it does not need. What no implementation may do is claim an authenticity it did not check (Section 8.9).

Partial claims are not a weakness to be tolerated but the mechanism that makes selective installation meaningful: a client that only reads never needs validation, contradiction detection, or a packing format, and requiring it to implement them to be conforming would push implementers toward claiming nothing at all.

15.2 Golden vectors

Agreement on the identity layer cannot be established by prose, because every disagreement in it is silent: two implementations that serialize differently do not fail, they simply compute different identifiers for the same knowledge and stop sharing blocks. Conformance is therefore anchored in published test vectors, covering the nine places where a divergence would be invisible:

- **Serialization:** values and the exact canonical bytes they produce, including the cases that MUST be rejected, an object key outside the Basic Multilingual Plane (where code-unit and code-point order differ), an NFC/NFD pair yielding two distinct blocks, and a duplicate-key document (Section 6.2).
- **Block identifiers:** envelopes and the `block_id` each one hashes to, including a provenance record that names assisting parties and the same record without them, which must keep the identifier it had before version 2 existed.
- **Actor identifiers:** identifiers of both forms and the rejections each with its reason, so that the rule of Section 5 is checkable without running any implementation.
- **Schema selection:** payloads and the `schema_version` the oldest-that-fits rule assigns them against the registered set, including a payload that satisfies several registered schemas, and the disjoint case of a version that removes a required field, where a payload carrying both members or neither MUST be refused (Section 6.9).

- **Merkle roots:** leaf sets and the root the construction of Section 6.3 produces, including the empty set, odd-sized trees, where a wrong split is otherwise easy to miss, a permuted composition that **MUST** be rejected, and a duplicate-carrying input with the sorted-unique root it collapses to.
- **Inclusion proofs:** proofs, each carrying its leaf index and tree size, and the roots they must reconstruct, for a tree of size one, a power-of-two size, and an odd size.
- **SSHSIG framing:** signature blobs and the exact bytes they parse to, including the signed-data blob, which is what separates a framing bug from a signing bug, and the malformed blobs that **MUST** be rejected.
- **Signatures:** snapshots, published test key pairs, namespaces, and the verdict a verifier **MUST** reach, including a signature valid under the wrong namespace, a key authorized at one chain position and retired at another, a key with a `compromised_from` record, a trust-root revision that fails the quorum rule, a signature whose `hash_algorithm` is not `sha512`, and inputs on the Ed25519 acceptance boundary, a non-canonical encoding and a small-order public key, both of which **MUST** be rejected (Section 8).
- **Reconciliation:** an ancestor composition and two branch compositions, with the set Equation 2 must produce, including the case where one side dropped what the other retained, and a criss-cross history with two best common ancestors whose reconciliation **MUST** be refused (Section 13.2).

These vectors **MUST** be plain data, readable without executing any implementation of this protocol. An implementation in another language demonstrates agreement by reproducing them, which is a stronger claim than passing a test suite supplied by a reference implementation, because it requires no dependency on that implementation at all.

The corpus is published independently of any implementation, at <https://github.com/gaussia-labs/boltzmann-conformance>, together with the schema registry companion of Section 6.9. The separation is not administrative. A corpus an implementation owns is governed by that implementation's layout and release cycle, and "conforming" decays into "matching that implementation, bugs included" – which would make the paragraph above a claim the corpus could not honour about itself. Corpus versions are tied to protocol versions, and a published vector is never edited: a case whose expected output would change is either a defect in whatever produced it or a new serialization identifier, and silently correcting one would let two implementations agree with the corpus at different times and disagree with each other.

Behavioral requirements that cannot be reduced to fixed vectors, such as the guarantee that a query returns only verified blocks, or that a drop yields a composition excluding the dropped identity, are stated as the **MUST** clauses of the sections that define them. A reference implementation **MAY** publish an executable suite for these; conformance is defined by the clauses, not by the suite.

16 Conclusion and Future Work

We presented Boltzmann Brain, a conceptual architecture for portable, verifiable, and model-agnostic knowledge. Its core commitment is a clean separation of concerns: the brain conserves, validates, and retrieves, while an external, interchangeable LLM interprets and contextualizes. From this commitment follow the five-module memory model, content-addressed blocks versioned by Merkle DAGs, derived indices, the three levels of hashes, OCI Artifacts as the distribution model, and an account of how knowledge leaves a brain.

Beyond the architecture, this version fixes the interoperability surface, from the block envelope to a conformance kit anchored in golden vectors.

It then addresses the two gaps an earlier version named as open, and in each case the existing structure did most of the work.

Authenticity was a question of where, not of how much. Integrity is already total: any consumer, offline and unconfigured, can confirm that a brain is exactly the brain its identifiers describe. What no amount of hashing supplies is authorship, because internal consistency is cheap to manufacture. Section 8 adds that one assertion in one place: a snapshot already commits to every module root and, through its parents, to the entire prior history, so a single detached signature over it covers everything. The authorized keys travel inside the brain so verification stays offline, their scopes are the protocol's own operations, revising the list requires a quorum from the previous revision, and validity is decided by position in the chain rather than by a forgeable timestamp. What cannot come from inside is trust at first contact.

Divergence was a question of what to record, not of what to compute. Section 13 makes parents a list, which is the whole structural change. A module composition is a set of immutable blocks, so no textual conflict is representable and the structural reconciliation is set arithmetic; a snapshot is a complete statement of composition rather than a patch, so merge, rebase and squash produce the same blocks and differ only in the lineage they record, which once snapshots are signed turns the choice into a question of attribution. The conflicts that remain are semantic, and re-validating the reconciled result as an ingestion turns each into a validation verdict instead of an unresolvable state. A contribution from outside then needs no new object: it is a signed snapshot, in another repository, whose parent belongs to the maintainer's history.

Section 14 states the adversarial model both of those imply, including the exposure the architecture creates by design: preserved evidence is routed into an external model, so a poisoned source that survives ingestion is replayed with the protocol's own verification stamp attached. Verification means that content is authentically what was committed, never that it is true or safe to act on.

What the protocol still owes. Eight questions remain, smaller and better specified than the two just closed.

- **Freshness.** Everything verifies offline, so a hostile registry cannot fabricate state, but it can withhold from a consumer that has seen nothing newer. Serving a returning consumer an ancestor of what it already holds is a rollback and is refused (Section 7.4), so what remains open is first contact. Signing does not fix this. The known answers, expiring metadata [52] and an append-only transparency log [29], both require a party outside the artifact, which is why neither is adopted here without deciding what to trade for it. This is the largest of the six.
- **Governance recovery.** When the govern quorum equals the number of holders, one compromised or lost key freezes governance permanently (Section 8.4). The margin rule reduces the odds; a recovery path that does not reopen self-assertion, perhaps social or out-of-band re-anchoring with consumer consent, is unexplored.
- **Confidentiality and selective disclosure.** Everything in a brain is readable by whoever can pull it. Encrypted blocks, read control, and selective disclosure are unaddressed, and crypto-shredding stays out of the removal table until an encryption scheme exists for it to act on (Section 8.13).
- **Removal from a travelling index.** The protocol now requires an index to be bound to its composition, and requires a stale one to be omitted or reported rather than shipped. It

does not define deletion of a single entry, because deletion from an approximate nearest-neighbour structure is engine-specific and lossy. Whether a protocol-level removal is worth specifying, given that omission is always available, is open.

- **A deterministic path for partial evidence loss.** A derived block that loses one of several sources cannot survive, because its citations are inside its identity. Rebuilding it currently costs a full round trip through the external model even though the surviving evidence has not changed, and a cheaper deterministic path for that specific case is undefined.
- **Propagation of erasure.** A redaction destroys bytes locally. The protocol can publish the fact of a removal, but a distributed brain can only signal destruction, not guarantee it across copies already pulled.
- **Decay in episodic memory.** Which function governs demotion is left to an implementation, and it is not obvious that it should be.
- **Federation across trust boundaries.** Reconciliation assumes one governed history. Nothing yet says how two independently governed brains, with different trust roots, cite each other's evidence without one of them adopting the other's authorities.

Validating all of this against real, multi-source brains remains the natural next step, and the properties added here are the ones that make such a validation meaningful beyond a single author: a brain that several parties extend is exactly the case where integrity was never the hard part.

References

- [1] Carlisle Adams, Pat Cain, Denis Pinkas, and Robert Zuccherato. Internet X.509 public key infrastructure time-stamp protocol (TSP). Technical Report RFC 3161, Internet Engineering Task Force, 2001. <https://www.rfc-editor.org/rfc/rfc3161>.
- [2] Anthropic. Model context protocol: Architecture overview. Model Context Protocol Specification, 2024. <https://modelcontextprotocol.io>.
- [3] Elaine Barker and Allen Roginsky. Transitioning the use of cryptographic algorithms and key lengths. Technical Report NIST SP 800-131A Rev. 2, National Institute of Standards and Technology, 2019. <https://doi.org/10.6028/NIST.SP.800-131Ar2>.
- [4] Robert A. Bjork and Elizabeth L. Bjork. A new theory of disuse and an old theory of stimulus fluctuation. In *From Learning Processes to Cognitive Processes: Essays in Honor of William K. Estes*, volume 2, pages 35–67. Erlbaum, 1992.
- [5] Scott Bradner. Key words for use in RFCs to indicate requirement levels. Technical Report RFC 2119, Internet Engineering Task Force, 1997. <https://www.rfc-editor.org/rfc/rfc2119>.
- [6] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2 edition, 2014. Chapter 10, Git Internals.
- [7] Konstantinos Chalkias, François Garillot, and Valeria Nikolaenko. Taming the many EdDSAs. In *Security Standardisation Research (SSR)*, 2020. Measures sixteen Ed25519 implementations disagreeing on signature validity. <https://eprint.iacr.org/2020/1244>.
- [8] Coalition for Content Provenance and Authenticity. C2PA technical specification, version 2.4. <https://spec.c2pa.org/>, 2026.

- [9] Cognee. The cognify operation. Cognee Documentation, 2025. <https://docs.cognee.ai>.
- [10] Cognee. Core concepts overview. Cognee Documentation, 2025. <https://docs.cognee.ai>.
- [11] Ronald L. Davis and Yi Zhong. The biology of forgetting: A perspective. *Neuron*, 95(3): 490–503, 2017.
- [12] Henry de Valence. ZIP 215: Explicitly defining and modifying Ed25519 validation rules. Zcash Improvement Proposal, 2020. <https://zips.z.cash/zip-0215>.
- [13] L. Peter Deutsch. DEFLATE compressed data format specification version 1.3. Technical Report RFC 1951, Internet Engineering Task Force, 1996. <https://www.rfc-editor.org/rfc/rfc1951>.
- [14] L. Peter Deutsch. GZIP file format specification version 4.3. Technical Report RFC 1952, Internet Engineering Task Force, 1996. <https://www.rfc-editor.org/rfc/rfc1952>.
- [15] Git project. `git-merge-base(1)`. Git documentation, 2024. “When the history involves criss-cross merges, there can be more than one best common ancestor... it is unspecified which best one is output.” <https://git-scm.com/docs/git-merge-base>.
- [16] Travis D. Goode, Kazumasa Z. Tanaka, Amar Sahay, and Thomas J. McHugh. The hippocampal engram as a memory index. *Neurobiology of Learning and Memory*, 2018.
- [17] Google Cloud. Introducing the open knowledge format. Google Cloud Blog, 2026. <https://cloud.google.com/blog>.
- [18] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.
- [19] Paul Groth, Andrew Gibson, and Jan Velterop. The anatomy of a nanopublication. *Information Services & Use*, 30(1–2):51–56, 2010.
- [20] in-toto project. in-toto attestation framework: Envelope specification. <https://github.com/in-toto/attestation/blob/main/spec/v1/envelope.md>, 2024.
- [21] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (EdDSA). Technical Report RFC 8032, Internet Engineering Task Force, 2017. <https://www.rfc-editor.org/rfc/rfc8032>.
- [22] Keybase. Keybase server documentation: Sigchains. <https://book.keybase.io/docs/server>, 2019. “Every link is checked against the state of the account at that point in the sigchain, old links remain valid even if their signing keys are revoked later.”.
- [23] Richard Kissel, Andrew Regenscheid, Matthew Scholl, and Kevin Stine. Guidelines for media sanitization. Technical Report NIST Special Publication 800-88 Revision 1, National Institute of Standards and Technology, 2014. Defines cryptographic erase.
- [24] James J. Knierim and Joshua P. Neunuebel. Tracking the flow of hippocampal computation: Pattern separation, pattern completion, and attractor dynamics. *Neurobiology of Learning and Memory*, 129:38–49, 2016.
- [25] Tobias Kuhn and Michel Dumontier. Trusty URIs: Verifiable, immutable, and permanent digital artifacts for linked data. In *The Semantic Web: Trends and Challenges (ESWC)*, 2014. arXiv:1401.5775.

- [26] Tobias Kuhn, Christine Chichester, Michael Krauthammer, Núria Queralt-Rosinach, Ruben Verborgh, George Giannakopoulos, Axel-Cyrille Ngonga Ngomo, Raffaele Vigiante, and Michel Dumontier. Decentralized provenance-aware publishing with nanopublications. *PeerJ Computer Science*, 2:e78, 2016.
- [27] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.
- [28] John E. Laird, Allen Newell, and Paul S. Rosenbloom. SOAR: An architecture for general intelligence. *Artificial Intelligence*, 33(1):1–64, 1987.
- [29] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. Technical Report RFC 9162, Internet Engineering Task Force, 2021. Defines the Merkle Tree Hash; obsoletes RFC 6962. <https://www.rfc-editor.org/rfc/rfc9162>.
- [30] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. PROV-O: The PROV ontology. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-o/>.
- [31] Microsoft. Graphrag: Indexing architecture and methods. Microsoft GraphRAG Documentation, 2024. <https://microsoft.github.io/graphrag>.
- [32] Microsoft. Graphrag: Query engine overview. Microsoft GraphRAG Documentation, 2024. <https://microsoft.github.io/graphrag>.
- [33] Alistair Miles and Sean Bechhofer. SKOS simple knowledge organization system reference. W3C Recommendation, 2009. Concept schemes with broader and narrower relations. <https://www.w3.org/TR/skos-reference/>.
- [34] MITRE. CVE-2012-2459: Merkle tree second-preimage ambiguity. Common Vulnerabilities and Exposures, 2012. Duplicating the last node on an odd level lets two distinct leaf sets produce the same root. <https://www.cve.org/CVERecord?id=CVE-2012-2459>.
- [35] John X. Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M. Rush. Text embeddings reveal (almost) as much as text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12448–12460, 2023.
- [36] T. Munirathinam. memorywire: A vendor-neutral wire format for agent memory operations. arXiv:2606.01138, 2026. v1, 31 May 2026. <https://arxiv.org/abs/2606.01138>.
- [37] Karim Nader and Oliver Hardt. A single standard for memory: The case for reconsolidation. *Nature Reviews Neuroscience*, 10(3):224–234, 2009.
- [38] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367, 2022. Signatures published as separate objects referring to the artifact they cover.
- [39] Open Container Initiative. Oci descriptor specification. OCI Specifications, 2024. <https://github.com/opencontainers/image-spec>.
- [40] Open Container Initiative. Oci image manifest specification. OCI Specifications, 2024. <https://github.com/opencontainers/image-spec>.
- [41] Open Container Initiative. OCI distribution specification: Listing referrers. Specification, 2024. <https://github.com/opencontainers/distribution-spec>.

- [42] OpenSSH. PROTOCOL.sshsig: Signing and verifying data with SSH keys. OpenSSH protocol documentation, 2019. Detached signatures over arbitrary data under a caller-chosen namespace. <https://github.com/openssh/openssh-portable/blob/master/PROTOCOL.sshsig>.
- [43] OpenSSH project. OpenSSH key revocation lists (PROTOCOL.krl). <https://raw.githubusercontent.com/openssh/openssh-portable/master/PROTOCOL.krl>, 2024.
- [44] ORAS. Understanding oci artifacts. ORAS Documentation, 2024. <https://oras.land>.
- [45] Rosa C. Paolicelli, Giulia Bolasco, Francesca Pagani, Laura Maggi, Maria Scianni, Patrizia Panzanelli, Maurizio Giustetto, Tiago Alves Ferreira, Eva Guiducci, Laura Dumas, Davide Ragozzino, and Cornelius T. Gross. Synaptic pruning by microglia is necessary for normal brain development. *Science*, 333(6048):1456–1458, 2011.
- [46] S. R. Ranganathan. *Prolegomena to Library Classification*. Asia Publishing House, Bombay, 3rd edition, 1967. Faceted classification: one hierarchy cannot serve every access need, so a document takes coordinates on several independent axes.
- [47] S. K. Ravindran. Portable agent memory: A protocol for provenance-verified memory transfer across heterogeneous LLM agents. arXiv:2605.11032, 2026. v1, 10 May 2026. <https://arxiv.org/abs/2605.11032>.
- [48] Nelson Rebola, Mario Carta, and Christophe Mulle. Operation and plasticity of hippocampal ca3 circuits: Implications for memory encoding. *Nature Reviews Neuroscience*, 18(4):208–220, 2017.
- [49] Roger L. Redondo and Richard G. M. Morris. Making memories last: The synaptic tagging and capture hypothesis. *Nature Reviews Neuroscience*, 12(1):17–30, 2011.
- [50] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON canonicalization scheme (JCS). Technical Report RFC 8785, Internet Engineering Task Force, 2020. <https://www.rfc-editor.org/rfc/rfc8785>.
- [51] Tomas J. Ryan and Paul W. Frankland. Forgetting as a form of adaptive engram cell plasticity. *Nature Reviews Neuroscience*, 23(3):173–186, 2022.
- [52] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*, pages 61–72, 2010. The Update Framework: threshold-signed root metadata, role separation, and expiring timestamps.
- [53] Secure Systems Lab. DSSE: Dead simple signing envelope. <https://github.com/secure-systems-lab/dsse>, 2023. `background.md` records the rationale for avoiding canonicalization as a signature dependency.
- [54] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. arXiv:2309.02427.
- [55] Garry Tan. Brain vs memory vs session. GBrain, GitHub repository, 2026. <https://github.com/garrytan/gbrain>.
- [56] Garry Tan. Pluggable engine architecture. GBrain, GitHub repository, 2026. <https://github.com/garrytan/gbrain>.

- [57] Garry Tan. Two-repo architecture: Agent behavior vs world knowledge. GBrain, GitHub repository, 2026. <https://github.com/garrytan/gbrain>.
- [58] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bytes and bytes. In *28th USENIX Security Symposium*, pages 1393–1410, 2019.
- [59] TUF community. The update framework specification. <https://theupdateframework.github.io/specification/latest/>, 2024. Root rotation: version N+1 must be signed by thresholds of both version N and version N+1.
- [60] Endel Tulving. Episodic and semantic memory. In Endel Tulving and Wayne Donaldson, editors, *Organization of Memory*, pages 381–403. Academic Press, 1972.
- [61] Michael A. Yassa and Craig E. L. Stark. Pattern separation in the hippocampus. *Trends in Neurosciences*, 34(10):515–525, 2011.
- [62] Tatu Ylonen and Chris Lonvick. The secure shell (SSH) protocol architecture. Technical Report RFC 4251, Internet Engineering Task Force, 2006. Section 4.1 states the first-contact key problem and the leap-of-faith model this document adopts for bootstrap. <https://www.rfc-editor.org/rfc/rfc4251>.
- [63] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. arXiv:2402.07867, 2024. <https://arxiv.org/abs/2402.07867>.